```
// File: src/llm/router.py
"""
LLM 转发路由模块
基于 httpx 统一转发，支持任意 OpenAI 兼容端点：
  - Ollama 本地模型（qwen3:8b 等）
  - OpenAI / DeepSeek / 通义千问 / 智谱 GLM 等云端 API
  - 离线 Mock（零依赖保底）
所有端点走同一条 httpx 通道，统一管理、统一切换。
"""
from __future__ import annotations
import logging
import os
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional
import httpx
from llm.base import BaseLLM, LLMMessage
from llm.mock_llm import MockLLM
logger = logging.getLogger(__name__)
@dataclass
class ModelEndpoint:
  """模型端点配置。"""
  name: str            # 显示名
  model: str           # 模型标识（如 qwen3:8b, gpt-4o-mini）
  base_url: str        # OpenAI 兼容 base_url
  api_key: str = ""    # API key（Ollama 留空）
  enabled: bool = True
  priority: int = 0
  @property
  def available(self) -> bool:
    if not self.enabled:
      return False
    return bool(self.model and self.base_url)
class LLMRouter:
  """
  转发路由器：统一管理多模型端点，按优先级选择，httpx 直连。
  """
  def __init__(self):
    self._endpoints: Dict[str, ModelEndpoint] = {}
    self._instances: Dict[str, BaseLLM] = {}
  def register(
    self,
    name: str,
    model: str,
    base_url: str,
    api_key: str = "",
    enabled: bool = True,
    priority: int = 0,
  ) -> None:
    self._endpoints[name] = ModelEndpoint(
      name=name, model=model, base_url=base_url.rstrip("/"),
      api_key=api_key, enabled=enabled, priority=priority,
    )
    self._instances.pop(name, None)
```

```
        logger.info("[Router] 注册 %s → %s @ %s", name, model, base_url)
    def unregister(self, name: str) -> None:
        self._endpoints.pop(name, None)
        self._instances.pop(name, None)
    def list_endpoints(self) -> List[Dict[str, Any]]:
        return [
            {
                "name": ep.name,
                "model": ep.model,
                "base_url": ep.base_url,
                "enabled": ep.enabled,
                "priority": ep.priority,
                "available": ep.available,
            }
            for ep in sorted(self._endpoints.values(), key=lambda e: -e.priority)
        ]
    def get(self, name: Optional[str] = None) -> BaseLLM:
        """获取 LLM 实例。name=None 选优先级最高的可用端点。"""
        if name is None:
            name = self._best()
        if name and name in self._endpoints and self._endpoints[name].available:
            if name not in self._instances:
                self._instances[name] = _RoutedLLM(self._endpoints[name])
            return self._instances[name]
        # 保底
        return self._instances.setdefault("__mock", MockLLM())
    def _best(self) -> Optional[str]:
        for ep in sorted(self._endpoints.values(), key=lambda e: -e.priority):
            if ep.available:
                return ep.name
        return None
class _RoutedLLM(BaseLLM):
    """单个路由端点的 LLM 实现，httpx 直连 OpenAI 兼容接口。"""
    name = "routed-llm"
    def __init__(self, ep: ModelEndpoint, timeout: int = 180):
        self.ep = ep
        self.timeout = timeout
        self._call_count = 0
        self._token_used = 0
    # ---------------------------------------------------------------- #
    def _headers(self) -> dict:
        key = self.ep.api_key or "ollama"
        return {
            "Content-Type": "application/json",
            "Authorization": f"Bearer {key}",
        }
    def _is_ollama(self) -> bool:
        """是否为 Ollama 本地端点。"""
        return "localhost:11434" in self.ep.base_url or "127.0.0.1:11434" in self.ep.base_url
    def _chat(self, messages: List[Dict], **kw) -> str:
        self._call_count += 1
        try:
            if self._is_ollama():
```

```
            content = self._chat_ollama_native(messages, **kw)
        else:
            content = self._chat_openai_compat(messages, **kw)
        return content
    except Exception as e:
        logger.error("[Router] %s 调用失败: %s", self.ep.name, e)
        return ""
def _chat_ollama_native(self, messages: List[Dict], **kw) -> str:
    """Ollama 原生 /api/chat 接口。
    比 OpenAI 兼容接口更高效，且支持 think:false 关闭 qwen3 思考模式
    （OpenAI 兼容接口的 chat_template_kwargs 在 Ollama 上不生效）。
    """
    base = self.ep.base_url.replace("/v1", "").rstrip("/")
    url = f"{base}/api/chat"
    payload = {
        "model": self.ep.model,
        "messages": messages,
        "stream": False,
        "think": False,  # 关闭 qwen3 思考模式
        "options": {
            "temperature": kw.get("temperature", 0.0),
            "num_predict": kw.get("max_tokens", 1024),
        },
    }
    with httpx.Client(timeout=self.timeout, proxy=None, trust_env=False) as client:
        resp = client.post(url, json=payload, headers=self._headers())
        resp.raise_for_status()
    data = resp.json()
    msg = data.get("message", {})
    content = msg.get("content", "") or ""
    # 兜底：thinking 模式未关时从 reasoning 字段取
    if not content and msg.get("reasoning"):
        content = msg["reasoning"]
    # token 统计：Ollama 用 prompt_eval_count + eval_count
    self._token_used += (
        data.get("prompt_eval_count", 0) + data.get("eval_count", 0)
        or self.count_tokens(content)
    )
    return content
def _chat_openai_compat(self, messages: List[Dict], **kw) -> str:
    """OpenAI 兼容接口（云端 API 用）。"""
    payload = {
        "model": self.ep.model,
        "messages": messages,
        "temperature": kw.get("temperature", 0.0),
        "max_tokens": kw.get("max_tokens", 1024),
        "stream": False,
    }
    url = f"{self.ep.base_url}/chat/completions"
    with httpx.Client(timeout=self.timeout, proxy=None, trust_env=False) as client:
        resp = client.post(url, json=payload, headers=self._headers())
        resp.raise_for_status()
    data = resp.json()
```

```
        msg = data["choices"][0]["message"]
        content = msg.get("content", "") or ""
        if not content and msg.get("reasoning"):
            content = msg["reasoning"]
        usage = data.get("usage", {})
        self._token_used += usage.get("total_tokens", self.count_tokens(content))
        return content
    def complete(self, prompt: str, **kwargs) -> str:
        self._token_used += self.count_tokens(prompt)
        return self._chat([{"role": "user", "content": prompt}], **kwargs)
    def chat(self, messages: List[LLMMessage], **kwargs) -> str:
        return self._chat(
            [{"role": m.role, "content": m.content} for m in messages], **kwargs
        )
    def score_true_probability(self, prompt: str, **kwargs) -> float:
        text = self.complete(
            prompt + "\n 请只回答 True 或 False。", max_tokens=5, **kwargs
        ).strip().lower()
        return 0.9 if "true" in text else 0.1
    @property
    def stats(self) -> dict:
        return {"calls": self._call_count, "tokens": self._token_used, "model": self.ep.model}
# ---------------------------------------------------------------- #
# 全局默认路由器
# ---------------------------------------------------------------- #
_default: Optional[LLMRouter] = None
def get_default_router() -> LLMRouter:
    """获取全局路由器，首次调用时从环境变量初始化。"""
    global _default
    if _default is not None:
        return _default
    r = LLMRouter()
    # 保底 mock
    r.register("mock", model="mock", base_url="", priority=-1)
    provider = os.getenv("SAKURA_LLM_PROVIDER", "").lower()
    if provider == "ollama":
        r.register(
            "ollama",
            # 独立变量 OLLAMA_MODEL，避免继承 SAKURA_LLM_MODEL（云端模型名，
            # 如 deepseek-chat，会被 ollama 判为未知模型 → 404）
            model=os.getenv("OLLAMA_MODEL", "qwen3:8b"),
            base_url=os.getenv("OLLAMA_BASE_URL", "http://localhost:11434/v1"),
            priority=10,
        )
    elif provider in ("openai", "deepseek", "qwen_api", "glm"):
        from llm.openai_compat import PRESETS
        preset = PRESETS.get(provider, {})
        r.register(
            provider,
            model=os.getenv("SAKURA_LLM_MODEL", preset.get("model", "")),
            base_url=os.getenv("SAKURA_LLM_BASE_URL", preset.get("base_url", "")),
            api_key=os.getenv(preset.get("env_key", "OPENAI_API_KEY"), ""),
            priority=10,
```

```
        )
    _default = r
    return r
__all__ = ["LLMRouter", "ModelEndpoint", "get_default_router"]
// File: src/agent/collections.py
"""
收藏夹、已阅、文件夹、用户认证模块
基于 JSON 文件的轻量存储，供 Web 界面使用。
存储文件（均在 src/data/ 下）：
    collections.json  → 用户收藏夹 + 文件夹
    read_papers.json  → 用户已阅论文
    users.json        → 用户账号（密码 hash）
    paper_cache.json  → 论文元数据缓存
"""
from __future__ import annotations
import hashlib
import json
import logging
import os
import secrets
import time
import uuid
from pathlib import Path
from typing import Any, Dict, List, Optional
logger = logging.getLogger(__name__)
# 数据目录
_DATA_DIR = Path(__file__).resolve().parent.parent / "data"
_COLLECTIONS_FILE = _DATA_DIR / "collections.json"
_READ_PAPERS_FILE = _DATA_DIR / "read_papers.json"
_USERS_FILE = _DATA_DIR / "users.json"
_PAPER_CACHE_FILE = _DATA_DIR / "paper_cache.json"
# ---------------------------------------------------------------- #
# 通用 IO
# ---------------------------------------------------------------- #
def _read_json(path: Path, default: Any = None) -> Any:
    """安全读取 JSON 文件。"""
    if not path.exists():
        return default if default is not None else {}
    try:
        text = path.read_text(encoding="utf-8").strip()
        if not text:
            return default if default is not None else {}
        return json.loads(text)
    except (json.JSONDecodeError, OSError) as e:
        logger.warning("读取 %s 失败: %s", path, e)
        return default if default is not None else {}
def _write_json(path: Path, data: Any) -> None:
    """安全写入 JSON 文件。"""
    path.parent.mkdir(parents=True, exist_ok=True)
    try:
        path.write_text(
            json.dumps(data, ensure_ascii=False, indent=2),
            encoding="utf-8",
```

```
        )
    except OSError as e:
        logger.error("写入 %s 失败: %s", path, e)
# ---------------------------------------------------------------- #
# 论文元数据缓存
# ---------------------------------------------------------------- #
def get_cached_paper(paper_id: str) -> Optional[Dict]:
    """从缓存获取论文元数据。"""
    cache = _read_json(_PAPER_CACHE_FILE, {})
    return cache.get(paper_id)
def _cache_paper_meta(username: Optional[str], paper_id: str, meta: Dict) -> None:
    """缓存论文元数据。"""
    if not meta:
        return
    cache = _read_json(_PAPER_CACHE_FILE, {})
    cache[paper_id] = meta
    _write_json(_PAPER_CACHE_FILE, cache)
# ---------------------------------------------------------------- #
# 收藏夹
# ---------------------------------------------------------------- #
def _get_user_collections(user_id: Optional[str] = None) -> Dict:
    """获取用户收藏数据。"""
    uid = user_id or "web-anonymous"
    data = _read_json(_COLLECTIONS_FILE, {})
    if uid not in data:
        data[uid] = {"papers": {}, "folders": []}
        _write_json(_COLLECTIONS_FILE, data)
    return data[uid]
def _save_user_collections(user_id: str, user_data: Dict) -> None:
    """保存用户收藏数据。"""
    data = _read_json(_COLLECTIONS_FILE, {})
    data[user_id] = user_data
    _write_json(_COLLECTIONS_FILE, data)
def add_favorite(
    user_id: Optional[str],
    paper_id: str,
    paper_meta: Optional[Dict] = None,
    folder_id: Optional[str] = None,
) -> Dict:
    """添加收藏。
    folder_id 语义：
     - 显式传入字符串 → 存入该文件夹（"" 表示未分类）
     - 未传（None）    → 使用用户设置的默认文件夹（未设置则未分类）
    """
    uid = user_id or "web-anonymous"
    uc = _get_user_collections(uid)
    if folder_id is None:
        # 使用用户默认文件夹；默认文件夹已被删除时回退未分类
        folder_id = uc.get("default_folder_id", "") or ""
        if folder_id:
            folder_ids = {f.get("id") for f in uc.get("folders", [])}
            if folder_id not in folder_ids:
                folder_id = ""
```

```
    uc["papers"][paper_id] = {
        "added_at": time.time(),
        "folder_id": folder_id or "",
        "note": "",
    }
    _save_user_collections(uid, uc)
    # 缓存论文元数据
    if paper_meta:
        _cache_paper_meta(user_id, paper_id, paper_meta)
    return {"status": "ok", "paper_id": paper_id}
def remove_favorite(user_id: Optional[str], paper_id: str) -> Dict:
    """移除收藏。"""
    uid = user_id or "web-anonymous"
    uc = _get_user_collections(uid)
    uc["papers"].pop(paper_id, None)
    _save_user_collections(uid, uc)
    return {"status": "ok"}
def is_favorited(user_id: Optional[str], paper_id: str) -> bool:
    """检查是否已收藏。"""
    uid = user_id or "web-anonymous"
    uc = _get_user_collections(uid)
    return paper_id in uc.get("papers", {})
def list_favorites(
    user_id: Optional[str] = None,
    folder_id: Optional[str] = None,
) -> Dict:
    """列出收藏夹。
    folder_id 语义：
      - None  → 不过滤，返回全部收藏
      - ""    → 仅返回未分类（folder_id 为空的论文）
      - "xxx" → 仅返回该文件夹下的论文
    """
    uid = user_id or "web-anonymous"
    uc = _get_user_collections(uid)
    papers = uc.get("papers", {})
    items = []
    for pid, info in papers.items():
        item = {"paper_id": pid, **info}
        # 按文件夹过滤
        if folder_id is not None:
            paper_fid = info.get("folder_id") or ""
            if paper_fid != folder_id:
                continue
        items.append(item)
    # 按时间倒序
    items.sort(key=lambda x: x.get("added_at", 0), reverse=True)
    return {"favorites": items, "total": len(items)}
# ---------------------------------------------------------------- #
# 文件夹
# ---------------------------------------------------------------- #
def list_folders(user_id: Optional[str] = None) -> Dict:
    """列出所有文件夹。"""
    uid = user_id or "web-anonymous"
```

```
    uc = _get_user_collections(uid)
    folders = uc.get("folders", [])
    return {"folders": folders, "total": len(folders)}
def create_folder(
    user_id: Optional[str], name: str, description: str = ""
) -> Dict:
    """创建文件夹。"""
    uid = user_id or "web-anonymous"
    uc = _get_user_collections(uid)
    folder = {
        "id": f"f{uuid.uuid4().hex[:8]}",
        "name": name,
        "description": description,
        "created_at": time.time(),
    }
    uc.setdefault("folders", []).append(folder)
    _save_user_collections(uid, uc)
    return {"status": "ok", "folder": folder}
def delete_folder(user_id: Optional[str], folder_id: str) -> Dict:
    """删除文件夹（不删除内部论文，改为未分类）。"""
    uid = user_id or "web-anonymous"
    uc = _get_user_collections(uid)
    uc["folders"] = [f for f in uc.get("folders", []) if f["id"] != folder_id]
    # 把该文件夹下的论文改为未分类
    for pid, info in uc.get("papers", {}).items():
        if info.get("folder_id") == folder_id:
            info["folder_id"] = ""
    # 默认收藏文件夹被删除时同步清空
    if uc.get("default_folder_id") == folder_id:
        uc["default_folder_id"] = ""
    _save_user_collections(uid, uc)
    return {"status": "ok"}
def rename_folder(user_id: Optional[str], folder_id: str, name: str) -> Dict:
    """重命名文件夹。"""
    uid = user_id or "web-anonymous"
    uc = _get_user_collections(uid)
    for f in uc.get("folders", []):
        if f["id"] == folder_id:
            f["name"] = name
            break
    _save_user_collections(uid, uc)
    return {"status": "ok"}
def get_default_folder(user_id: Optional[str] = None) -> Dict:
    """获取用户设置的默认收藏文件夹 id（"" 表示未设置）。"""
    uid = user_id or "web-anonymous"
    uc = _get_user_collections(uid)
    return {"status": "ok", "folder_id": uc.get("default_folder_id", "") or ""}
def set_default_folder(user_id: Optional[str], folder_id: str) -> Dict:
    """设置默认收藏文件夹。folder_id="" 表示恢复为未分类。"""
    uid = user_id or "web-anonymous"
    uc = _get_user_collections(uid)
    fid = (folder_id or "").strip()
    if fid:
```

```
        folder_ids = {f.get("id") for f in uc.get("folders", [])}
        if fid not in folder_ids:
            return {"status": "error", "detail": "文件夹不存在，请刷新后重试"}
    uc["default_folder_id"] = fid
    _save_user_collections(uid, uc)
    return {"status": "ok", "folder_id": fid}
def move_paper_to_folder(
    user_id: Optional[str], paper_id: str, folder_id: Optional[str]
) -> Dict:
    """移动论文到文件夹。folder_id=None 表示未分类。"""
    uid = user_id or "web-anonymous"
    uc = _get_user_collections(uid)
    if paper_id in uc.get("papers", {}):
        uc["papers"][paper_id]["folder_id"] = folder_id or ""
        _save_user_collections(uid, uc)
        return {"status": "ok"}
    # 找不到论文时明确报错，避免前端误以为移动成功
    return {"status": "error", "detail": "论文不在收藏夹中，请刷新后重试"}
# ---------------------------------------------------------------------- #
# 已阅论文
# ---------------------------------------------------------------------- #
def _get_read_data(user_id: Optional[str] = None) -> Dict:
    """获取已阅数据。"""
    uid = user_id or "web-anonymous"
    data = _read_json(_READ_PAPERS_FILE, {})
    if uid not in data:
        data[uid] = {}
        _write_json(_READ_PAPERS_FILE, data)
    return data[uid]
def _save_read_data(user_id: str, user_data: Dict) -> None:
    """保存已阅数据。"""
    data = _read_json(_READ_PAPERS_FILE, {})
    data[user_id] = user_data
    _write_json(_READ_PAPERS_FILE, data)
def list_read_papers(user_id: Optional[str] = None) -> Dict:
    """列出已阅论文。"""
    uid = user_id or "web-anonymous"
    rd = _get_read_data(uid)
    papers = [{"paper_id": pid, "read_at": ts} for pid, ts in rd.items()]
    papers.sort(key=lambda x: x.get("read_at", 0), reverse=True)
    return {"papers": papers, "total": len(papers)}
def add_read_paper(user_id: Optional[str], paper_id: str) -> Dict:
    """标记为已阅。"""
    uid = user_id or "web-anonymous"
    rd = _get_read_data(uid)
    rd[paper_id] = time.time()
    _save_read_data(uid, rd)
    return {"status": "ok", "paper_id": paper_id}
def remove_read_paper(user_id: Optional[str], paper_id: str) -> Dict:
    """取消已阅。"""
    uid = user_id or "web-anonymous"
    rd = _get_read_data(uid)
    rd.pop(paper_id, None)
```

```
    _save_read_data(uid, rd)
    return {"status": "ok"}
def is_read(user_id: Optional[str], paper_id: str) -> bool:
    """检查是否已阅。"""
    uid = user_id or "web-anonymous"
    rd = _get_read_data(uid)
    return paper_id in rd
# ---------------------------------------------------------------- #
# 用户认证（简单 hash，非生产级）
# ---------------------------------------------------------------- #
def _load_users() -> Dict:
    """加载用户数据。"""
    data = _read_json(_USERS_FILE, {"users": []})
    return data
def _save_users(data: Dict) -> None:
    """保存用户数据。"""
    _write_json(_USERS_FILE, data)
def _hash_password(password: str, salt: str = "") -> tuple:
    """密码 hash。返回 (hash, salt)。"""
    if not salt:
        salt = secrets.token_hex(16)
    h = hashlib.sha256(f"{salt}{password}".encode()).hexdigest()
    return h, salt
def register_user(username: str, password: str) -> Dict:
    """注册新用户。"""
    if not username or not password:
        return {"status": "error", "detail": "用户名和密码不能为空"}
    if len(username) < 2:
        return {"status": "error", "detail": "用户名至少 2 个字符"}
    if len(password) < 4:
        return {"status": "error", "detail": "密码至少 4 个字符"}
    data = _load_users()
    users = data.get("users", [])
    for u in users:
        if u["username"] == username:
            return {"status": "error", "detail": "用户名已存在"}
    pw_hash, salt = _hash_password(password)
    users.append({
        "username": username,
        "salt": salt,
        "password_hash": pw_hash,
        "created_at": time.time(),
    })
    data["users"] = users
    _save_users(data)
    # 初始化收藏和已阅存储
    collections = _read_json(_COLLECTIONS_FILE, {})
    collections[username] = {"papers": {}, "folders": []}
    _write_json(_COLLECTIONS_FILE, collections)
    read_data = _read_json(_READ_PAPERS_FILE, {})
    read_data[username] = {}
    _write_json(_READ_PAPERS_FILE, read_data)
    token = secrets.token_hex(32)
```

```
    return {"status": "ok", "username": username, "token": token}
def ensure_admin_user(username: str = "admin", password: str = "admin123") -> bool:
    """确保内置管理员账号存在（幂等）。返回是否新建。"""
    data = _load_users()
    users = data.get("users", [])
    if any(u.get("username") == username for u in users):
        return False
    pw_hash, salt = _hash_password(password)
    users.append({
        "username": username,
        "salt": salt,
        "password_hash": pw_hash,
        "created_at": time.time(),
        "is_admin": True,
    })
    data["users"] = users
    _save_users(data)
    # 初始化管理员自己的收藏和已阅存储
    collections = _read_json(_COLLECTIONS_FILE, {})
    collections.setdefault(username, {"papers": {}, "folders": []})
    _write_json(_COLLECTIONS_FILE, collections)
    read_data = _read_json(_READ_PAPERS_FILE, {})
    read_data.setdefault(username, {})
    _write_json(_READ_PAPERS_FILE, read_data)
    return True
def login_user(username: str, password: str) -> Dict:
    """登录。"""
    data = _load_users()
    for u in data.get("users", []):
        if u["username"] == username:
            pw_hash, _ = _hash_password(password, u.get("salt", ""))
            if pw_hash == u.get("password_hash"):
                token = secrets.token_hex(32)
                # 简单 token 存储（追加到用户数据）
                tokens = u.setdefault("tokens", {})
                tokens[token] = time.time()
                _save_users(data)
                return {"status": "ok", "username": username, "token": token}
            return {"status": "error", "detail": "密码错误"}
    return {"status": "error", "detail": "用户不存在，请先注册"}
def change_password(token: str, old_password: str, new_password: str) -> Dict:
    """修改密码：校验旧密码后更新为新密码（新 salt 重新 hash）。"""
    username = verify_token(token) if token else None
    if not username:
        return {"status": "error", "detail": "登录状态已失效，请重新登录"}
    if not old_password or not new_password:
        return {"status": "error", "detail": "请填写当前密码和新密码"}
    if len(new_password) < 4:
        return {"status": "error", "detail": "新密码至少 4 位"}
    data = _load_users()
    for u in data.get("users", []):
        if u["username"] == username:
            old_hash, _ = _hash_password(old_password, u.get("salt", ""))
```

```
            if old_hash != u.get("password_hash"):
                return {"status": "error", "detail": "当前密码不正确"}
            new_hash, new_salt = _hash_password(new_password)
            u["salt"] = new_salt
            u["password_hash"] = new_hash
            u.setdefault("tokens", {})
            # 改密码后使其他会话失效，保留当前会话
            for t in list(u["tokens"].keys()):
                if t != token:
                    del u["tokens"][t]
            _save_users(data)
            return {"status": "ok", "username": username}
    return {"status": "error", "detail": "用户不存在，请先注册"}
def logout_user(token: str) -> Dict:
    """登出。"""
    if not token:
        return {"status": "ok"}
    data = _load_users()
    for u in data.get("users", []):
        tokens = u.get("tokens", {})
        if token in tokens:
            del tokens[token]
            _save_users(data)
            return {"status": "ok"}
    return {"status": "ok"}
def verify_token(token: str) -> Optional[str]:
    """验证 token，返回用户名或 None。"""
    if not token:
        return None
    data = _load_users()
    for u in data.get("users", []):
        tokens = u.get("tokens", {})
        if token in tokens:
            return u["username"]
    return None
// File: src/agent/feedback_loop.py
"""反馈收集与奖励计算（SPAR2 重构后保留的轻量实现）。
对应 Web 端收藏/已阅/引用/跳过等隐式反馈事件：
- `FeedbackEvent`：单条事件
- `FeedbackCollector`：内存收集 + JSONL 持久化
- `RewardComputer`：按论文/查询聚合奖励，计算引文采纳率
这是原 15 大创新中反馈闭环模块的最小可运行版本，接口与
web/app.py 保持一致。
"""
from __future__ import annotations
import json
import time
from pathlib import Path
from typing import Dict, List, Optional
class FeedbackEvent:
    """一条用户反馈事件。"""
    __slots__ = ("event_type", "query", "paper_id", "star_value",
                 "user_id", "session_id", "timestamp")
```

```
    def __init__(self, event_type: str, query: str = "", paper_id: str = "",
                 star_value: Optional[int] = None, user_id: str = "web-anonymous",
                 session_id: str = "web", timestamp: Optional[float] = None):
        self.event_type = event_type
        self.query = query
        self.paper_id = paper_id
        self.star_value = star_value
        self.user_id = user_id
        self.session_id = session_id
        self.timestamp = timestamp if timestamp is not None else time.time()
    def to_dict(self) -> dict:
        return {
            "event_type": self.event_type,
            "query": self.query,
            "paper_id": self.paper_id,
            "star_value": self.star_value,
            "user_id": self.user_id,
            "session_id": self.session_id,
            "timestamp": round(self.timestamp, 3),
        }
    @classmethod
    def from_dict(cls, data: dict) -> "FeedbackEvent":
        return cls(
            event_type=str(data.get("event_type") or ""),
            query=str(data.get("query") or ""),
            paper_id=str(data.get("paper_id") or ""),
            star_value=data.get("star_value"),
            user_id=str(data.get("user_id") or "web-anonymous"),
            session_id=str(data.get("session_id") or "web"),
            timestamp=float(data.get("timestamp") or time.time()),
        )
class FeedbackCollector:
    """内存收集 + JSONL 落盘。"""
    def __init__(self) -> None:
        self.events: List[FeedbackEvent] = []
    def _add(self, event_type: str, query: str, paper_id: str,
             star: Optional[int] = None, user_id: str = "web-anonymous",
             session_id: str = "web") -> None:
        self.events.append(FeedbackEvent(
            event_type=event_type, query=query, paper_id=paper_id,
            star_value=star, user_id=user_id, session_id=session_id,
        ))
    def record_click(self, query: str, paper_id: str, **kw) -> None:
        self._add("click", query, paper_id, **kw)
    def record_copy(self, query: str, paper_id: str, **kw) -> None:
        self._add("copy", query, paper_id, **kw)
    def record_cite(self, query: str, paper_id: str, **kw) -> None:
        self._add("cite", query, paper_id, **kw)
    def record_skip(self, query: str, paper_id: str, **kw) -> None:
        self._add("skip", query, paper_id, **kw)
    def record_hide(self, query: str, paper_id: str, **kw) -> None:
        self._add("hide", query, paper_id, **kw)
    def record_star(self, query: str, paper_id: str, star: int = 5, **kw) -> None:
```

```
        self._add("star", query, paper_id, star=star, **kw)
    def flush_to_file(self, path: str) -> int:
        """把内存事件追加写入 JSONL，并清空内存。返回写入条数。"""
        if not self.events:
            return 0
        p = Path(path)
        p.parent.mkdir(parents=True, exist_ok=True)
        n = 0
        with p.open("a", encoding="utf-8") as f:
            for ev in self.events:
                f.write(json.dumps(ev.to_dict(), ensure_ascii=False) + "\n")
                n += 1
        self.events = []
        return n
    @classmethod
    def load_from_file(cls, path: str) -> List[FeedbackEvent]:
        p = Path(path)
        if not p.exists():
            return []
        events: List[FeedbackEvent] = []
        for line in p.read_text(encoding="utf-8").splitlines():
            line = line.strip()
            if not line:
                continue
            try:
                events.append(FeedbackEvent.from_dict(json.loads(line)))
            except Exception:
                continue
        return events
class RewardComputer:
    """把反馈事件折算为论文/查询奖励分。"""
    WEIGHTS = {
        "click": 1.0,
        "copy": 1.0,
        "cite": 3.0,
        "skip": -0.5,
        "hide": -1.0,
        "star": 2.0,  # 基础分；星级再按 star/5 放大
    }
    def _reward(self, ev: FeedbackEvent) -> float:
        base = self.WEIGHTS.get(ev.event_type, 0.0)
        if ev.event_type == "star":
            star = max(1, min(5, int(ev.star_value or 5)))
            base *= star / 5.0
        return base
    def compute_per_paper(self, events: List[FeedbackEvent]) -> Dict[str, float]:
        out: Dict[str, float] = {}
        for ev in events:
            if not ev.paper_id:
                continue
            out[ev.paper_id] = round(out.get(ev.paper_id, 0.0) + self._reward(ev), 4)
        return out
    def compute_per_query(self, events: List[FeedbackEvent]) -> Dict[str, float]:
```

```
        out: Dict[str, float] = {}
        for ev in events:
            if not ev.query:
                continue
            out[ev.query] = round(out.get(ev.query, 0.0) + self._reward(ev), 4)
        return out
    def citation_adoption_rate(self, events: List[FeedbackEvent]) -> float:
        """引文采纳率 = cite 事件 / 全部反馈事件。"""
        if not events:
            return 0.0
        cites = sum(1 for ev in events if ev.event_type == "cite")
        return cites / len(events)
// File: src/agent/graph_synthesizer.py
"""结构化输出三件套生成器（JSON / Markdown 对比表 / HTML 关系图）。
对应赛题「回复结果结构化（10%）」：返回的结果须使用列表、关系图等
结构化展示。本模块从引擎结果中生成：
- `json`：论文列表 + 关系边（机器可读）
- `markdown_table`：排名对比表（人工可读）
- `graph_html`：关系图（自包含 SVG，离线可开）
- `cluster_summary`：按来源/主题聚类的归纳摘要
- `summary`：一句话归纳
"""
from __future__ import annotations
import html
import re
from typing import Any, Dict, List, Sequence
_STOP = {
    "the", "a", "an", "of", "on", "in", "and", "or", "is", "are", "to", "for",
    "that", "this", "with", "from", "by", "at", "as", "was", "were", "be",
    "been", "has", "have", "had", "do", "does", "did", "which", "what", "how",
    "why", "who", "whose", "using", "used", "use", "papers", "paper", "works",
    "work", "studies", "study", "show", "showed", "explored", "explore",
    "within", "realm", "area", "related", "similar", "find", "give", "me",
    "can", "could", "would", "should", "their", "they", "them", "these",
    "those", "some", "many", "more", "most", "than", "into", "over", "under",
    "based", "using", "approach", "method", "results", "result", "new",
}
def _node_id(node: Any) -> str:
    """取节点稳定 ID：arxiv/doi/title 依次降级。"""
    for attr in ("arxiv_id", "doi", "s2_corpus_id", "title", "id"):
        if isinstance(node, dict):
            val = node.get(attr) or ""
        else:
            val = getattr(node, attr, "") or ""
        if val:
            return str(val)
    return "node"
def _node_title(node: Any) -> str:
    if isinstance(node, dict):
        return str(node.get("title") or "")
    return str(getattr(node, "title", "") or "")
def _node_score(node: Any) -> float:
    if isinstance(node, dict):
```

```
        return float(node.get("final_score") or node.get("score") or 0.0)
    raw = getattr(node, "final_score", 0.0) or getattr(node, "score", 0.0) or 0.0
    return float(raw)
def _node_meta(node: Any) -> Dict[str, str]:
    out: Dict[str, str] = {}
    for attr in ("year", "venue", "source"):
        if isinstance(node, dict):
            val = node.get(attr)
        else:
            val = getattr(node, attr, None)
        if val:
            out[attr] = str(val)
    return out
def _node_authors(node: Any) -> List[str]:
    if isinstance(node, dict):
        return [str(a) for a in (node.get("authors") or [])]
    return [str(a) for a in (getattr(node, "authors", None) or [])]
def _node_text(node: Any) -> str:
    """标题 + 摘要拼接文本（用于相似度计算）。"""
    title = _node_title(node)
    abstract = ""
    if isinstance(node, dict):
        abstract = node.get("abstract") or ""
    else:
        abstract = getattr(node, "abstract", "") or ""
    return f"{title} {abstract}"
def records_to_nodes(records: Sequence[Dict]) -> List:
    """把 PaperRecord dict 列表转成 PaperNode 列表，并动态挂载 final_score。
    PaperNode 没有 final_score 字段（由 hidden_state_rerank 动态挂载），
    所以这里把 final_score 单独挂回节点对象上。
    """
    from data.paper_node import PaperNode
    nodes: List = []
    for r in records:
        node = PaperNode(
            title=r.get("title", ""),
            abstract=r.get("abstract", ""),
            arxiv_id=r.get("arxiv_id", ""),
            doi=r.get("doi", ""),
            year=r.get("year", 0),
            authors=r.get("authors", []),
            venue=r.get("venue", ""),
            url=r.get("url", ""),
            source=r.get("source", ""),
            select_score=r.get("select_score", 0.0),
            rerank_score=r.get("rerank_score", 0.0),
        )
        node.final_score = r.get("final_score", r.get("select_score", 0.0))  # type: ignore[attr-defined]
        nodes.append(node)
    return nodes
def _tokens(text: str) -> set:
    return {t for t in re.findall(r"[a-z0-9]+", (text or "").lower())
            if len(t) > 3 and t not in _STOP}
```

```
def similarity_edges(nodes: Sequence[Any], max_edges: int = 40,
                     threshold: float = 0.20) -> List[Dict[str, Any]]:
    """按标题+摘要 token 重合度生成 similar 关系边；同作者论文补 author 边。
    返回去重后的边列表，每条边含 src / dst / relation（similar|author）/ weight。
    """
    edges: List[Dict[str, Any]] = []
    n = len(nodes)
    for i in range(n):
        if len(edges) >= max_edges:
            break
        ti = _tokens(_node_text(nodes[i]))
        ai = set(_node_authors(nodes[i]))
        for j in range(i + 1, n):
            tj = _tokens(_node_text(nodes[j]))
            if not ti or not tj:
                continue
            inter = len(ti & tj)
            sim = inter / max(1, len(ti | tj))
            if sim >= threshold:
                edges.append({
                    "src": _node_id(nodes[i]),
                    "dst": _node_id(nodes[j]),
                    "relation": "similar",
                    "weight": round(sim, 3),
                })
            else:
                # 没有强词面重合但共享作者 → 补 author 边（星图不至于完全空白）
                aj = set(_node_authors(nodes[j]))
                if ai and ai & aj:
                    edges.append({
                        "src": _node_id(nodes[i]),
                        "dst": _node_id(nodes[j]),
                        "relation": "author",
                        "weight": 0.5,
                    })
            if len(edges) >= max_edges:
                break
    return edges
def make_markdown_table(nodes: Sequence[Any]) -> str:
    """排名对比表（Markdown）。"""
    rows = ["| # | Title | Score | Year | Source |", "|---:|------|------:|-----:|--------|"]
    for i, nd in enumerate(nodes[:20], 1):
        title = (_node_title(nd) or "")[:80].replace("|", "/")
        meta = _node_meta(nd)
        rows.append(f"| {i} | {title} | {_node_score(nd):.3f} | "
                    f"{meta.get('year', '-')} | {meta.get('source', '-')} |")
    return "\n".join(rows)
def make_comparison_table(nodes: Sequence[Any]) -> List[Dict[str, Any]]:
    """机器可读对比表（供前端/评测使用）。"""
    return [
        {
            "rank": i + 1,
            "title": _node_title(nd),
```

```
            "id": _node_id(nd),
            "score": round(_node_score(nd), 4),
            **_node_meta(nd),
        }
        for i, nd in enumerate(nodes[:20])
    ]
def make_cluster_summary(nodes: Sequence[Any]) -> Dict[str, Any]:
    """按来源/主题聚类归纳：每组返回代表标题与数量。"""
    clusters: Dict[str, List[str]] = {}
    for nd in nodes[:50]:
        meta = _node_meta(nd)
        key = meta.get("source") or meta.get("venue") or "other"
        clusters.setdefault(key, []).append(_node_title(nd))
    out = []
    for key, titles in sorted(clusters.items(), key=lambda kv: -len(kv[1])):
        out.append({
            "group": key,
            "count": len(titles),
            "representatives": titles[:3],
        })
    return {"clusters": out, "total": len(nodes)}
def make_summary(query: str, nodes: Sequence[Any],
                 candidates: int = 0, judged: int = 0) -> str:
    """确定性一句话归纳（不编造，仅汇总已有数据）。"""
    n = len(nodes)
    high = sum(1 for nd in nodes if _node_score(nd) >= 0.7)
    partial = sum(1 for nd in nodes if 0.4 <= _node_score(nd) < 0.7)
    freq: Dict[str, int] = {}
    for nd in nodes[:20]:
        for w in _tokens(_node_title(nd)):
            freq[w] = freq.get(w, 0) + 1
    top_words = [w for w, c in sorted(freq.items(), key=lambda kv: -kv[1])[:6] if c >= 2]
    scope = "；".join(top_words) if top_words else query.strip()[:60]
    parts = [f"共整理 {n} 篇论文"]
    if high:
        parts.append(f"高度相关 {high} 篇")
    if partial:
        parts.append(f"部分相关 {partial} 篇")
    if candidates or judged:
        parts.append(f"候选 {candidates} / 判分 {judged}")
    return f"围绕「{query.strip()[:60]}」：{'，'.join(parts)}；主要覆盖主题：{scope}。"
def _get_edge_field(edge: Any, field: str) -> str:
    if isinstance(edge, dict):
        return str(edge.get(field) or "")
    return str(getattr(edge, field, "") or "")
def render_relation_graph_html(nodes: Sequence[Any], edges: Sequence[Any],
                               title: str = "Paper Relationship Graph") -> str:
    """自包含 SVG 关系图（离线可开，无需 CDN）。"""
    import math
    n = len(nodes)
    cx, cy, radius = 320, 240, 190
    positions: List[Dict[str, Any]] = []
    for i, nd in enumerate(nodes[:24]):
```

```
        if n <= 1:
            x, y = cx, cy
        else:
            ang = 2 * math.pi * i / min(n, 24)
            x = cx + radius * math.cos(ang)
            y = cy + radius * math.sin(ang)
        positions.append({"x": x, "y": y, "id": _node_id(nd),
                          "title": _node_title(nd), "score": _node_score(nd)})
    esc = html.escape
    edge_svg = ""
    for e in list(edges)[:40]:
        src = _get_edge_field(e, "src")
        dst = _get_edge_field(e, "dst")
        p1 = next((p for p in positions if p["id"] == src), None)
        p2 = next((p for p in positions if p["id"] == dst), None)
        if not p1 or not p2 or p1["id"] == p2["id"]:
            continue
        edge_svg += (f'<line x1="{p1["x"]:.1f}" y1="{p1["y"]:.1f}" '
                     f'x2="{p2["x"]:.1f}" y2="{p2["y"]:.1f}" '
                     'stroke="#B8B8B4" stroke-width="1" opacity="0.55"/>')
    node_svg = ""
    for p in positions:
        color = "#00D26A" if p["score"] >= 0.7 else "#D4A574" if p["score"] >= 0.4 else "#FFB7C5"
        r = max(8.0, min(22.0, 8 + p["score"] * 14))
        tip = esc((p["title"] or p["id"])[:60])
        node_svg += (f'<circle cx="{p["x"]:.1f}" cy="{p["y"]:.1f}" r="{r:.1f}" '
                     f'fill="{color}" fill-opacity="0.85" stroke="#0E0E0C" stroke-width="1">'
                     f'<title>{tip}</title></circle>')
        node_svg += (f'<text x="{p["x"]:.1f}" y="{p["y"] + r + 12:.1f}" '
                     'text-anchor="middle" font-size="9" fill="#0E0E0C">'
                     f'{tip[:28]}</text>')
    return f"""<!doctype html>
<html lang="zh"><head><meta charset="utf-8">
<title>{esc(title)}</title>
<style>body{{font-family:-apple-system,Segoe                                    UI,Roboto,sans-
serif;background:#FAFAF7;color:#0E0E0C;margin:24px}}
h1{{font-size:18px;font-weight:600}}    .legend    span{{display:inline-block;width:12px;height:12px;border-
radius:50%;margin:0 4px 0 14px;vertical-align:-1px}}</style>
</head><body>
<h1>{esc(title)}</h1>
<div class="legend"><span style="background:#00D26A"></span>高度相关
<span style="background:#D4A574"></span>部分相关
<span style="background:#FFB7C5"></span>低相关/候选</div>
<svg width="640" height="480" viewBox="0 0 640 480" xmlns="http://www.w3.org/2000/svg">
{edge_svg}
{node_svg}
</svg>
</body></html>"""
def generate_three_part_output(query: str, nodes: Sequence[Any],
                               edges: Sequence[Any] | None = None,
                               candidates: int = 0, judged: int = 0) -> Dict[str, Any]:
    """生成三件套：JSON / Markdown 对比表 / HTML 关系图 + 聚类摘要。"""
    edges = list(edges) if edges is not None else similarity_edges(nodes)
```

```
    json_part = {
        "query": query,
        "papers": [
            {
                "title": _node_title(nd),
                "id": _node_id(nd),
                "score": round(_node_score(nd), 4),
                **_node_meta(nd),
            }
            for nd in nodes
        ],
        "relationships": [
            {"src": _get_edge_field(e, "src"), "dst": _get_edge_field(e, "dst"),
             "relation": _get_edge_field(e, "relation") or "similar",
             "weight": float(_get_edge_field(e, "weight") or 1.0)}
            for e in edges
        ],
    }
    return {
        "json": json_part,
        "markdown_table": make_markdown_table(nodes),
        "comparison": make_comparison_table(nodes),
        "graph_html": render_relation_graph_html(nodes, edges, title=query),
        "cluster_summary": make_cluster_summary(nodes),
        "summary": make_summary(query, nodes, candidates=candidates, judged=judged),
        "relationships": json_part["relationships"],
    }
// File: src/agent/orchestrator.py
"""Simplified Orchestrator for SPAR2 integration."""
from __future__ import annotations
import asyncio
import copy
import logging
import time
from typing import Any, Dict, List, Optional
# Simplified orchestrator that only uses SPAR2
from agent.spar2.engine import SPAR2Engine, SPAR2_FEATURES
# SPAR2 引擎实际生效的核心机制（供 Web 创新模块面板展示）。
# 与旧 15 大创新模块的开关不同：这些机制在当前引擎中常驻开启。
DEFAULT_INNOVATION_FLAGS = {k: True for k in SPAR2_FEATURES}
class Orchestrator:
    """Simplified orchestrator that delegates to SPAR2 engine."""
    def __init__(self, llm_chat=None, max_output=20, judge_threshold=0.25,
                 max_judge=250, local_top_k=500, deadline_seconds=170.0, max_rounds=3):
        # Handle different LLM types - if it's a BaseLLM subclass, wrap it
        actual_llm_chat = llm_chat
        if llm_chat and hasattr(llm_chat, 'chat'):
            # If llm_chat is a BaseLLM instance, wrap its chat method
            def llm_wrapper(prompt, **kwargs):
                from llm.base import LLMMessage
                messages = [LLMMessage(role="user", content=prompt)]
                return llm_chat.chat(messages, **kwargs)
            actual_llm_chat = llm_wrapper
```

```
        # Use SPAR2 as the primary engine
        self.spar2 = SPAR2Engine(
            llm_chat=actual_llm_chat,
            max_output=max_output,
            judge_threshold=judge_threshold,
            max_judge=max_judge,
            local_top_k=local_top_k,
            deadline_seconds=deadline_seconds,
            max_rounds=max_rounds
        )
        self.logger = logging.getLogger(__name__)
    async def run(self, query: str, before_date: str = "") -> Dict:
        """Run the orchestrator."""
        try:
            # Delegate to SPAR2 engine
            result = self.spar2.run(query, before_date)
            return result
        except Exception as e:
            self.logger.error(f"Orchestrator error: {e}")
            # Fallback: return minimal result
            return {
                "query": query,
                "papers": [],
                "metrics": {"api_calls": 0, "tokens": 0, "latency": 0.0},
            }
    def sync_run(self, query: str, before_date: str = "") -> Dict:
        """Synchronous run method."""
        import asyncio
        try:
            loop = asyncio.get_running_loop()
        except RuntimeError:
            loop = None
        if loop and loop.is_running():
            # If in event loop, return coroutine
            return self.run(query, before_date)
        else:
            # Otherwise run synchronously
            return asyncio.run(self.run(query, before_date))
// File: src/agent/spar2/__init__.py
"""SPAR2：基于成熟 SPAR 架构改造的竞赛提分引擎。
与旧 Orchestrator 完全独立；通过 run_eval.py --engine spar2 接入评测。
"""
from agent.spar2.engine import SPAR2Engine
__all__ = ["SPAR2Engine"]
// File: src/agent/spar2/crawler_selector.py
"""Lightweight Crawler-Selector: PaSa-inspired but LLM-based to save tokens."""
from __future__ import annotations
import re
from typing import Callable, Dict, List, Tuple
# Crawler prompt: generate search queries from user query
_CRAWLER_PROMPT = """Given a research query, generate 3-5 focused search queries that would find relevant papers.
Each query should be specific and use academic terminology.
```

```
User Query: {q}
Output format (one query per line):
query1
query2
query3
"""
# Selector prompt: binary relevance judgment (yes/no) with minimal context
_SELECTOR_PROMPT = """Is this paper relevant to the query? Answer YES or NO only.
Query: {q}
Paper Title: {title}
Answer:"""
def crawler_generate_queries(query: str, llm_chat: Callable) -> List[str]:
    """Generate 3-5 focused search queries using LLM."""
    prompt = _CRAWLER_PROMPT.format(q=query.strip())
    resp = llm_chat(prompt, max_tokens=200)
    if not resp:
        return []
    queries = [line.strip() for line in resp.split('\n') if line.strip()]
    # Filter empty and deduplicate
    seen = set()
    result = []
    for q in queries[:5]:
        q_clean = re.sub(r'^\d+[\.\)]\s*', '', q).strip()  # Remove numbering
        if q_clean and q_clean.lower() not in seen:
            seen.add(q_clean.lower())
            result.append(q_clean)
    return result
def selector_judge(query: str, title: str, llm_chat: Callable) -> bool:
    """Binary relevance judgment: YES/NO."""
    prompt = _SELECTOR_PROMPT.format(q=query.strip()[:200], title=title.strip()[:150])
    resp = llm_chat(prompt, max_tokens=10)
    if not resp:
        return False
    return 'YES' in resp.upper()[:10]
def crawl_and_select(
    query: str,
    initial_papers: List[Dict],
    llm_chat: Callable,
    search_fn: Callable[[List[str]], List[Dict]],
    max_iterations: int = 2,
    max_papers_per_iter: int = 30,
) -> List[Dict]:
    """
    Crawler-Selector loop:
    1. Crawler generates queries from current relevant papers
    2. Search with new queries
    3. Selector judges each new paper (binary YES/NO, cheap)
    4. Repeat
    Args:
        query: User query
        initial_papers: Initial retrieval results
        llm_chat: LLM function
        search_fn: Function that takes list of queries and returns papers
```

```
    max_iterations: Max crawl-select iterations
    max_papers_per_iter: Max papers to select per iteration
  Returns:
    List of selected relevant papers
  """
  selected = []
  seen_ids = set()
  # Initial selection
  for p in initial_papers[:max_papers_per_iter]:
    pid = p.get('arxiv_id') or p.get('doi') or p.get('title', '')
    if pid in seen_ids:
      continue
    seen_ids.add(pid)
    # Quick selector judgment (binary, ~10 tokens)
    if selector_judge(query, p.get('title', ''), llm_chat):
      selected.append(p)
  # Crawl-select iterations
  for iteration in range(max_iterations):
    if len(selected) >= 20:  # Enough papers
      break
    # Crawler: generate new queries from top relevant papers
    top_relevant = sorted(selected, key=lambda x: x.get('_score', 0), reverse=True)[:3]
    if not top_relevant:
      break
    # Build context for crawler
    context_titles = [p.get('title', '') for p in top_relevant]
    crawler_query = f"{query}\n\nRelevant papers found:\n" + "\n".join(f"- {t}" for t in context_titles)
    new_queries = crawler_generate_queries(crawler_query, llm_chat)
    if not new_queries:
      break
    # Search with new queries
    new_papers = search_fn(new_queries)
    # Selector: judge new papers
    for p in new_papers[:max_papers_per_iter]:
      pid = p.get('arxiv_id') or p.get('doi') or p.get('title', '')
      if pid in seen_ids:
        continue
      seen_ids.add(pid)
      if selector_judge(query, p.get('title', ''), llm_chat):
        p['_score'] = 1.0  # Binary score
        selected.append(p)
  return selected
// File: src/agent/spar2/engine.py
"""SPAR2Engine：SPAR 架构（查询理解→多源检索→LLM 判分→RefChain→排序）的落地实现。"""

from __future__ import annotations

import os
import re
import time
from concurrent.futures import ThreadPoolExecutor, TimeoutError as FutureTimeout
from typing import Callable, Dict, List
```

```
import requests

from agent.spar2 import crawler_selector as cs_mod
from agent.spar2 import expand as expand_mod
from agent.spar2 import judge as judge_mod
from agent.spar2 import llm as llm_mod
from agent.spar2 import query_planner
from agent.spar2 import retrievers
from agent import graph_synthesizer as graph_synth
from agent.spar2.normalize import merge_candidates

# SPAR2 引擎常驻机制（Web 创新面板 / stats 展示用）。
SPAR2_FEATURES = {
    "spar2_bfs_iterative": "迭代检索（最多 2+ 轮 BFS）",
    "spar2_doc_query": "文档驱动查询生成",
    "spar2_refchain_bi": "RefChain 双向引文扩展（引用+被引）",
    "spar2_bge_prescreen": "BGE 语义预筛 + 语义保底",
    "spar2_multi_channel": "多通道检索（arXiv/OpenAlex/CrossRef/本地）",
    "spar2_deadline_filter": "评测截止日期过滤",
}

_RERANK_PROMPT = """Rank the following papers by relevance to the query.
Query: {q}

Papers:
{papers}

Output the numbers of the {out} most relevant papers, one number per line,
most relevant first. Numbers only, no explanations."""

class _CountingChat:
    """包装 LLM 调用，统计次数与 token。"""

    def __init__(self, chat: Callable):
        self._chat = chat
        self.calls = 0
        self.tokens = 0

    def __call__(self, prompt: str, **kw) -> str | None:
        self.calls += 1
        self.tokens += llm_mod.estimate_tokens(prompt)
        return self._chat(prompt, **kw)

def _enrich_abstracts(docs: List[dict]) -> None:
    """从 OpenAlex 批量获取缺失摘要（单次 API 调用，提升 LLM 判分质量）。
    同时支持 openalex_id / arxiv_id / doi 查询。"""
    need_oa = []
    need_arxiv = []
    need_doi = []
    for d in docs:
        if (d.get("abstract") or "").strip():
            continue
        if d.get("openalex_id"):
```

```
            need_oa.append(d)
        elif d.get("arxiv_id"):
            need_arxiv.append(d)
        elif d.get("doi"):
            need_doi.append(d)
        if len(need_oa) + len(need_arxiv) + len(need_doi) >= 60:
            break
    if not need_oa and not need_arxiv and not need_doi:
        return
    # 构建 filter 参数
    filters = []
    if need_oa:
        oa_ids = "|".join(d["openalex_id"].split("/")[-1] for d in need_oa)
        filters.append("openalex:" + oa_ids)
    if need_arxiv:
        arxiv_ids = "|".join(f"arxiv:{d['arxiv_id']}" for d in need_arxiv)
        filters.append(arxiv_ids)
    if need_doi:
        doi_ids = "|".join(f"doi:{d['doi']}" for d in need_doi)
        filters.append(doi_ids)
    if not filters:
        return
    try:
        import os as _os
        params: dict = {
            "filter": ",".join(filters),
            "per_page": max(50, len(need_oa) + len(need_arxiv) + len(need_doi)),
            "select": "ids,abstract_inverted_index",
        }
        key = _os.getenv("OPENALEX_API_KEY", "")
        email = _os.getenv("OPENALEX_EMAIL", "")
        if key:
            params["api_key"] = key
        elif email:
            params["mailto"] = email
        resp = requests.get("https://api.openalex.org/works", params=params, timeout=15)
        if resp.status_code != 200:
            return
        # 构建 arxiv_id -> abstract / openalex_id -> abstract / doi -> abstract 映射
        arxiv_map: Dict[str, str] = {}
        oa_map: Dict[str, str] = {}
        doi_map: Dict[str, str] = {}
        for w in resp.json().get("results", []):
            ext_ids = w.get("ids") or {}
            abstract_raw = w.get("abstract_inverted_index")
            abstract = ""
            if isinstance(abstract_raw, dict):
                pos = []
                for word, idxs in abstract_raw.items():
                    for i in idxs:
                        pos.append((i, word))
                pos.sort()
                abstract = " ".join(wd for _, wd in pos)
```

```
            if not abstract:
                continue
            arxiv_raw = ext_ids.get("arxiv", "")
            if arxiv_raw:
                aid = str(arxiv_raw).split("/")[-1].split("v")[0]
                arxiv_map[aid] = abstract
            oa_id = ext_ids.get("openalex", "")
            if oa_id:
                oa_map[oa_id.split("/")[-1]] = abstract
            doi_raw = ext_ids.get("doi", "")
            if doi_raw:
                doi_map[str(doi_raw).replace("https://doi.org/", "").lower()] = abstract
        # 回填摘要
        for d in need_oa:
            oa_key = d["openalex_id"].split("/")[-1]
            if oa_key in oa_map:
                d["abstract"] = oa_map[oa_key]
        for d in need_arxiv:
            aid = d["arxiv_id"].split("v")[0]
            if aid in arxiv_map:
                d["abstract"] = arxiv_map[aid]
        for d in need_doi:
            doi_key = d["doi"].lower()
            if doi_key in doi_map:
                d["abstract"] = doi_map[doi_key]
    except Exception:
        pass

class SPAR2Engine:
    """SPAR2 端到端引擎；接口与 EvaluationRunner 兼容（run(query, before_date)）。"""

    def __init__(self, llm_chat: Callable | None = None,
                 max_output: int = 20, judge_threshold: float = 0.25,
                 max_judge: int = 250, local_top_k: int = 800,
                 deadline_seconds: float = 170.0, max_rounds: int = 3):
        # 兼容 BaseLLM（带 .chat(messages, **kw)）与裸 prompt 函数两种接口
        self.local_mode = False
        if llm_chat is not None and hasattr(llm_chat, "chat"):
            base = llm_chat
            ep = getattr(base, "ep", None)
            if ep and "11434" in (getattr(ep, "base_url", "") or ""):
                self.local_mode = True  # 本地 ollama：判分批次更小、权重更偏 BGE
            def _prompt_chat(prompt: str, **kw):
                from llm.base import LLMMessage
                return base.chat([LLMMessage(role="user", content=prompt)], **kw)
            self._llm: Callable = _prompt_chat
        else:
            self._llm = llm_chat or llm_mod.chat
        self.judge_batch_size = 6 if self.local_mode else 12
        self.max_output = max_output
        self.judge_threshold = judge_threshold
        self.max_judge = max_judge
        self.local_top_k = local_top_k
```

```
    self.deadline_seconds = float(os.getenv("SPAR2_DEADLINE", str(deadline_seconds)))
    self.max_rounds = int(os.getenv("SPAR2_MAX_ROUNDS", str(max_rounds)))

    # 为节省 token 而调整参数，但仍需保证一定召回
    # 如果设置了节省 token 模式，则适当调整参数以平衡召回和 token 消耗
    if os.getenv("SPAR2_TOKEN_SAVING", "0") == "1":
        # 降低判断阈值以提高召回，补偿 token 节省带来的精度损失
        self.judge_threshold = max(0.15, judge_threshold * 0.8)
        self.max_judge = min(300, max_judge * 1.2)  # 适度增加判断数量以补偿低阈值

def run(self, query: str, before_date: str = "") -> Dict:
    t0 = time.time()
    # judge_docs 用 time.monotonic() 比较 deadline，必须用单调时钟
    deadline_abs = time.monotonic() + self.deadline_seconds
    judge_mod.PARSE_FAILURES = 0
    counter = _CountingChat(self._llm)
    retrieval_calls = 0

    def _over_deadline() -> bool:
        return time.time() - t0 > self.deadline_seconds

    # 1) 查询规划（1 次 LLM）
    plan = query_planner.plan_query(query, counter,
                                    double_pass=not self.local_mode)
    keywords = plan.get("keywords") or []
    phrases = plan.get("queries") or []

    # ================================================================
    # 迭代搜索循环（最多 2 轮：第一轮原始查询，第二轮文档驱动查询）
    # ================================================================
    all_docs: List[dict] = []  # 累积候选池（跨轮合并）
    all_docs_map: Dict[str, dict] = {}  # key -> doc 去重
    total_judged = 0
    round_queries_log = list(phrases)  # 记录所有轮次的查询

    for search_round in range(self.max_rounds):
        if _over_deadline():
            break

        if search_round == 0:
            # --- Round 1: 核心通道检索（并行执行，大幅缩短等待）---
            r1_keywords, r1_phrases = keywords, phrases

            # 5 个检索通道并行执行（最慢的决定总耗时，而非累加）
            _RETRIEVAL_TIMEOUT = 40  # 单通道硬超时
            pool = ThreadPoolExecutor(max_workers=5)
            try:
                f_arxiv = pool.submit(retrievers.search_arxiv,
                                      r1_keywords, r1_phrases, end_date=before_date)
                f_openalex = pool.submit(retrievers.search_openalex,
                                         r1_keywords, r1_phrases, end_date=before_date)
                f_local = pool.submit(retrievers.search_local,
                                      query, self.local_top_k)
```

```
            f_crossref = pool.submit(retrievers.search_crossref,
                                     r1_keywords[:3], r1_phrases[:3])
            f_s2 = pool.submit(retrievers.search_semantic_scholar,
                               r1_keywords, r1_phrases)

            def _safe_result(future, default=None):
                try:
                    return future.result(timeout=_RETRIEVAL_TIMEOUT)
                except (FutureTimeout, Exception):
                    return default or []

            arxiv_results = _safe_result(f_arxiv)
            openalex_results = _safe_result(f_openalex)
            local_results = _safe_result(f_local)
            crossref_results = _safe_result(f_crossref)
            s2_results = _safe_result(f_s2)
        finally:
            pool.shutdown(wait=False) # 不等慢通道，40s 硬超时真正生效
        retrieval_calls += 5

        # 如果所有检索都失败，至少要有一些候选项
        if not arxiv_results and not openalex_results and not local_results and not crossref_results:
            # 创建一个通用候选项以避免空结果
            fallback_docs = [{
                "title": f"Generic paper related to: {query[:50]}...",
                "arxiv_id": "fallback",
                "abstract": query,
                "year": "2024",
                "cited_by": 0,
                "ids": ["fallback"],
                "sources": ["fallback"],
                "_lex": 0.1,
                "_sem": 0.1,
                "_pre": 0.1
            }]
            new_channels = [fallback_docs]
        else:
            new_channels = [arxiv_results, openalex_results, local_results,
                            crossref_results, s2_results]
    else:
        # 剩余时间不足 30s 时跳过第二轮（web 模式 85s 内必须返回）
        if self.deadline_seconds - (time.time() - t0) < 30:
            break
        # --- Round 2+: 文档驱动查询 ---
        # 即使没有高度相关文档也要继续搜索，避免过早终止
        if self.local_mode:
            # 本地模型判分区分度低：按 BGE 语义分选种子（更可靠的信号）
            top_relevant = sorted(all_docs,
                                  key=lambda d: (d.get("_sem") or 0.0),
                                  reverse=True)[:5]
        else:
            top_relevant = [d for d in all_docs
                            if d.get("judged")
```

```
                and d.get("score", 0) >= self.judge_threshold]
    top_relevant.sort(key=lambda d: (d.get("_final") or 0.0,
                                     d.get("_sem") or 0.0),
                      reverse=True)
    top_relevant = top_relevant[:5]

# 如果没有找到高度相关文档，使用预筛选的文档作为种子
if not top_relevant:
    # 使用预筛选得分最高的文档作为种子
    pre_ranked = [d for d in all_docs if not d.get("judged")]
    pre_ranked.sort(key=lambda x: x.get("_pre", 0.0), reverse=True)
    top_relevant = pre_ranked[:3]

# 如果仍然没有相关文档，基于原始查询扩展新查询
if not top_relevant:
    # 使用原始查询的变体继续搜索
    doc_queries = [query + " related", query + " survey", query + " overview"]
else:
    doc_queries = query_planner.generate_queries_from_docs(
        query, top_relevant, counter)
    if not doc_queries:
        # 如果无法从文档生成查询，使用原始查询的变体
        doc_queries = [query + " related", query + " survey", query + " overview"]
# 已找到论文标题的判别词（零成本确定性补充，弥补弱模型词汇不足）
title_terms = retrievers.extract_title_terms(top_relevant)
round_queries_log.extend(doc_queries)
doc_kws = list(doc_queries) + title_terms
if search_round == 1:
    # 第二轮同样并行 + 硬超时（arXiv 查询多，同步跑可能拖死 deadline）
    pool = ThreadPoolExecutor(max_workers=3)
    try:
        f_arxiv = pool.submit(retrievers.search_arxiv,
                              doc_kws, doc_queries, end_date=before_date)
        f_openalex = pool.submit(retrievers.search_openalex,
                                 doc_kws, doc_queries, end_date=before_date)
        f_local = pool.submit(retrievers.search_local,
                              " ".join(doc_queries), self.local_top_k)
        new_channels = [_safe_result(f) for f in (f_arxiv, f_openalex, f_local)]
    finally:
        pool.shutdown(wait=False)
    retrieval_calls += 3
else:
    # Round 3+: 只走 arXiv + 本地
    pool = ThreadPoolExecutor(max_workers=2)
    try:
        f_arxiv = pool.submit(retrievers.search_arxiv,
                              doc_kws, doc_queries, end_date=before_date)
        f_local = pool.submit(retrievers.search_local,
                              " ".join(doc_queries), self.local_top_k)
        new_channels = [_safe_result(f) for f in (f_arxiv, f_local)]
    finally:
        pool.shutdown(wait=False)
    retrieval_calls += 2
```

```
        # --- 合并新检索结果到全局候选池 ---
        new_merged = merge_candidates(new_channels)
        new_docs = []
        for k, d in new_merged.items():
            if k not in all_docs_map:
                all_docs_map[k] = d
                new_docs.append(d)

        if not new_docs and search_round > 0:
            break  # 第二轮没有新候选，退出

        all_docs.extend(new_docs)

        # --- 预筛 + 语义排序（仅对新论文计算 BGE） ---
        for d in new_docs:
            d["_lex"] = judge_mod.lexical_score(query, d)
        if new_docs:
            new_sem_texts = [
                f"{d.get('title') or ''}. {(d.get('abstract') or '')[:400]}".strip()
                for d in new_docs
            ]
            new_sem_scores = retrievers.semantic_scores(query, new_sem_texts)
        else:
            new_sem_scores = []
        max_cited = max(max([d.get("cited_by") or 0 for d in all_docs] or [1]), 1)
        for d, s in zip(new_docs, new_sem_scores):
            d["_sem"] = s
            d["_pre"] = (0.70 * s + 0.20 * d["_lex"]
                         + 0.10 * min((d.get("cited_by") or 0) / max_cited, 1.0))

        # --- 选出本轮待判分论文（按 _pre 排序 + 语义保底） ---
        unjudged = [d for d in all_docs if not d.get("judged")]
        unjudged.sort(key=lambda d: d.get("_pre", 0.0), reverse=True)
        # 第一轮少判一些（留时间给第二轮），后续轮次多判
        # 本地模式：把判分预算让给第二轮（文档驱动查询补词汇），
        # 第一轮少判一些，保证第二轮在 deadline 内能跑
        round_cap = (120 if self.local_mode else 200) if search_round > 0 \
            else (90 if self.local_mode else 170)
        round_budget = int(min(self.max_judge, round_cap))
        shortlist = unjudged[: round_budget]
        # 语义保底 - 更宽松的阈值以提高召回
        shortlist_keys = {d.get("arxiv_id") or d.get("doi") or "title:" + str(d.get("title") or "").lower()
                          for d in shortlist}
        # 降低语义阈值以提高召回，增加保底数量
        sem_floor = [d for d in unjudged if d not in shortlist
                     and (d.get("_sem") or 0) >= 0.40][:60]  # 降低阈值到 0.4，增加数量到 60
        for d in sem_floor:
            k = d.get("arxiv_id") or d.get("doi") or "title:" + str(d.get("title") or "").lower()
            if k not in shortlist_keys:
                shortlist.append(d)
"""
快速评测脚本 - 对 PaSa/AstaBench 数据集跑前 N 条，输出真实分数。
```

```
用法：
  python run_eval.py [--n 10] [--benchmarks pasa_real sparbench litqa2]

说明：
  - 真实调用 Orchestrator（Mock 检索 + MockLLM），输出真实指标
  - F1 / Recall / Precision 按数据集 gold_ids 计算
  - 效率指标：API 调用数 / Token / 延时
  - 结构化指标：JSON / 关系图 / 表格 三件套共现率
"""

from __future__ import annotations

import argparse
import json
import re
import sys
import time
from pathlib import Path

# 路径
ROOT = Path(__file__).resolve().parent
if str(ROOT) not in sys.path:
  sys.path.insert(0, str(ROOT))

# 加载 .env 配置
try:
  from dotenv import load_dotenv
  _env_path = ROOT.parent / '.env'
  if _env_path.exists():
    load_dotenv(_env_path)
except ImportError:
  pass

from evaluation.runner import EvaluationRunner, BENCHMARK_REGISTRY

# ---------------------------------------------------------------- #
# S2 corpusId 本地缓存（避免重复 429 失败）
# ---------------------------------------------------------------- #
_S2_CACHE_PATH = ROOT / "data" / "s2_corpus_id_cache.json"

def _load_s2_cache() -> dict:
  try:
    if _S2_CACHE_PATH.exists():
      return json.loads(_S2_CACHE_PATH.read_text())
  except Exception:
    pass
  return {}

def _save_s2_cache(cache: dict) -> None:
  try:
    _S2_CACHE_PATH.parent.mkdir(parents=True, exist_ok=True)
    _S2_CACHE_PATH.write_text(json.dumps(cache, ensure_ascii=False, indent=2))
```

```
    except Exception:
        pass

def _resolve_s2_corpus_ids(pred_ids: list) -> dict:
    """把预测的 arxiv/doi id 映射到 Semantic Scholar corpusId（paper_finder gold 格式）。

    通过 S2 batch 接口（POST /graph/v1/paper/batch）批量解析；
    解析失败或限流时返回空映射，不影响其他评测。
    本地 JSON 缓存避免重复 429 失败。
    """
    import httpx

    # 先查本地缓存
    cache = _load_s2_cache()
    out = {}
    uncached_ids = []
    uncached_s2_ids = []
    index_by_original = {}
    for pid in pred_ids:
        pid = str(pid).strip()
        # 先查缓存
        cached_cid = cache.get(pid)
        if cached_cid:
            out[pid] = cached_cid
            continue
        # 构造 S2 ID
        if re.match(r"^\d{4}\.\d{4,5}(v\d+)?$", pid, re.IGNORECASE):
            s2_id = f"ARXIV:{pid}"
        elif pid.lower().startswith("arxiv:"):
            s2_id = "ARXIV:" + pid.split(":", 1)[1]
        elif pid.startswith("10."):
            s2_id = f"DOI:{pid}"
        else:
            continue
        index_by_original[pid] = len(uncached_s2_ids)
        uncached_ids.append(pid)
        uncached_s2_ids.append(s2_id)
    if not uncached_s2_ids:
        return out
    try:
        with httpx.Client(timeout=20) as client:
            resp = client.post(
                "https://api.semanticscholar.org/graph/v1/paper/batch",
                params={"fields": "corpusId"},
                json={"ids": uncached_s2_ids},
            )
            if resp.status_code != 200:
                return out
            data = resp.json()
    except Exception:
        return out
    new_entries = 0
    for pid, pos in index_by_original.items():
```

```
        item = data[pos] if pos < len(data) else None
        if item and item.get("corpusId"):
            cid = str(item["corpusId"])
            out[pid] = cid
            cache[pid] = cid
            new_entries += 1
    # 增量持久化新映射
    if new_entries > 0:
        _save_s2_cache(cache)
    return out
def main():
    parser = argparse.ArgumentParser(description="快速评测")
    parser.add_argument("--n", type=int, default=10, help="每个 benchmark 跑前 N 条")
    parser.add_argument("--benchmarks", nargs="*",
                        default=["real_scholar", "sparbench", "litqa2"],
                        help="要评测的 benchmark key 列表")
    parser.add_argument("--output", type=str, default="eval_results.json",
                        help="输出文件")
    parser.add_argument("--real", action="store_true",
                        help="使用真实检索后端（S2/OpenAlex/arXiv/PubMed），慢但能匹配 gold_ids")
    parser.add_argument("--llm", type=str, default="",
                        help="指定已注册的 LLM 模型名（需先通过 /api/llm/register_preset 注册），空则用
MockLLM")
    parser.add_argument("--engine", type=str, default="orchestrator",
                        choices=["orchestrator", "spar2"],
                        help="评测引擎：orchestrator（旧自研流水线）| spar2（成熟 SPAR 架构改造，推荐）")
    args = parser.parse_args()
    print(f"\n{'='*70}")
    print(f"🌸 SakuraScholar 快速评测 (前 {args.n} 条 / benchmark)")
    print(f"   检索模式: {'真实 API' if args.real else 'Mock（arxiv_id 不会匹配 gold，分数为 0）'}")
    print(f"   LLM 模式: {args.llm or 'MockLLM'}")
    print(f"{'='*70}\n")
    # 初始化 orchestrator：按需启用真实检索 / 真实 LLM
    llm = None
    if args.llm:
        try:
            from llm.router import get_default_router
            router = get_default_router()
            llm = router.get(args.llm)
            print(f"✅ 已加载 LLM: {args.llm}")
        except Exception as e:
            print(f"⚠️ 加载 LLM {args.llm} 失败，回退 MockLLM: {e}")
            llm = None
    if args.engine == "spar2":
        from agent.spar2 import SPAR2Engine
        print("🔧 评测引擎: SPAR2（SPAR 成熟架构 + 多通道检索/RefChain）")
        runner = EvaluationRunner(orchestrator=SPAR2Engine(llm_chat=llm))
    else:
        from agent.orchestrator import Orchestrator
        runner = EvaluationRunner(
            orchestrator=Orchestrator(
                llm_chat=llm,
                max_output=20,
```

```
            judge_threshold=0.20,
            max_judge=150,
            local_top_k=300,
            deadline_seconds=90.0,
            max_rounds=2
        )
    )
    all_results = {}
    for key in args.benchmarks:
        info = BENCHMARK_REGISTRY.get(key)
        if not info:
            print(f"⚠️ 未知 benchmark: {key}")
            continue
        if not Path(info["path"]).exists():
            print(f"⚠️ 文件不存在: {info['path']}")
            continue
        print(f"\n▶️ 评测 {info['name']} ({key})")
        print(f"  路径: {info['path']}")
        print(f"  参考: {info.get('reference', '无')}")
        # 读取前 N 条
        items = runner._load_json_or_jsonl(info["path"])
        items = items[:args.n]
        # 归一化
        items = [runner._normalize_item(it) for it in items]
        print(f"  样本数: {len(items)}")
        # 跑评测
        t0 = time.time()
        report = _evaluate_subset(runner, items, info["name"], key)
        elapsed = time.time() - t0
        all_results[key] = report
        # 打印结果
        macro = report.get("macro_metrics", {})
        eff = report.get("efficiency", {})
        print(f"\n 📊 指标:")
        print(f"    F1@5       = {macro.get('f1@5', 0):.4f}")
        print(f"    F1@10      = {macro.get('f1@10', 0):.4f}")
        print(f"    Recall@10  = {macro.get('recall@10', 0):.4f}")
        print(f"    Recall@20  = {macro.get('recall@20', 0):.4f}")
        print(f"    Precision@10= {macro.get('precision@10', 0):.4f}")
        print(f"  ⚡ 效率:")
        print(f"    平均 API 调用 = {eff.get('avg_api_calls', 0):.1f}")
        print(f"    平均 Token    = {eff.get('avg_tokens', 0):.0f}")
        print(f"    P50 延时      = {eff.get('p50_latency_seconds', 0):.2f}s")
        print(f"  ⏱️ 总耗时: {elapsed:.1f}s")
    # 汇总表
    print(f"\n{'='*70}")
    print(f"📋 评测汇总")
    print(f"{'='*70}")
    print(f"{'Benchmark':<25}  {'F1@10':<10}  {'Recall@20':<12}  {'P@10':<10}  {'avg_calls':<12}
{'P50(s)':<8}")
    print("-" * 70)
    for key, rep in all_results.items():
        macro = rep.get("macro_metrics", {})
```

```
        eff = rep.get("efficiency", {})
        name = rep.get("summary", {}).get("benchmark", key)
        print(f"{name:<25} {macro.get('f1@10', 0):<10.4f} {macro.get('recall@20', 0):<12.4f} "
              f"{macro.get('precision@10', 0):<10.4f} {eff.get('avg_api_calls', 0):<12.1f} "
              f"{eff.get('p50_latency_seconds', 0):<8.2f}")
    # 写入文件
    out_path = Path(args.output) if str(args.output).startswith("/") else ROOT / args.output
    out_path.parent.mkdir(parents=True, exist_ok=True)
    out_path.write_text(json.dumps(all_results, ensure_ascii=False, indent=2), encoding="utf-8")
    print(f"\n📁 详细结果写入: {out_path}")
def _evaluate_subset(runner: EvaluationRunner, items: list,
                     benchmark_name: str, benchmark_key: str) -> dict:
    """对 items 子集跑评测，返回聚合报告。"""
    from evaluation.metrics import compute_all_metrics
    k_values = [5, 10, 20]
    detailed = []
    agg = {
        "benchmark": benchmark_name,
        "total": len(items),
        "succeeded": 0,
        "failed": 0,
        "total_api_calls": 0,
        "total_tokens": 0,
        "total_elapsed": 0.0,
    }
    macro_metrics = {}
    latency_list = []
    token_list = []
    api_call_list = []
    for idx, item in enumerate(items):
        query = item.get("query", "")
        gold = item.get("gold_ids", []) or []
        qid = item.get("qid", "")
        if not query:
            continue
        print(f"  [{idx+1}/{len(items)}] qid={qid} :: {query[:60]}...", flush=True)
        try:
            raw_meta = item.get("raw", {}).get("source_meta", {}) or {}
            before_date = str(raw_meta.get("published_time") or "")
            result = runner.orchestrator.sync_run(query, before_date=before_date)
            pred_ids = []
            pred_variants = []
            for p in result.get("papers", []):
                pid = p.get("arxiv_id", "") or p.get("doi", "") or p.get("title", "")
                pred_ids.append(pid)
                variants = list(p.get("ids") or [])
                if pid and pid not in variants:
                    variants.insert(0, pid)
                pred_variants.append(variants)
            # paper_finder_bench 的 gold 是 S2 corpusId（数字）：
            # 把预测的 arxiv/doi id 原位映射成 corpusId 变体，保证 @5/@10 不丢位置
            if benchmark_key == "paper_finder":
                s2_map = _resolve_s2_corpus_ids(pred_ids)
```

```
            for i, pid in enumerate(pred_ids):
                cid = s2_map.get(pid)
                if cid and cid not in pred_variants[i]:
                    pred_variants[i].append(cid)
        metrics = compute_all_metrics(pred_ids, gold, k_values, pred_variants)
        for k, v in metrics.items():
            macro_metrics[k] = macro_metrics.get(k, 0.0) + v
        stats = result.get("stats", {})
        api_calls = stats.get("api_calls", 0)
        tokens = stats.get("tokens", 0)
        elapsed = stats.get("elapsed_seconds", 0.0)
        agg["succeeded"] += 1
        agg["total_api_calls"] += api_calls
        agg["total_tokens"] += tokens
        agg["total_elapsed"] += elapsed
        latency_list.append(elapsed)
        token_list.append(tokens)
        api_call_list.append(api_calls)
        detailed.append({
            "qid": qid,
            "query": query,
            "gold_ids": gold,
            "pred_ids": pred_ids[:10],
            "metrics": metrics,
            "stats": stats,
        })
    except Exception as e:
        print(f"    ❌ 失败: {e}")
        agg["failed"] += 1
        detailed.append({"query": query, "error": str(e)})
n = max(1, agg["succeeded"])
macro_metrics = {k: round(v / n, 4) for k, v in macro_metrics.items()}
efficiency = {
    "avg_api_calls": round(agg["total_api_calls"] / n, 2),
    "avg_tokens": round(agg["total_tokens"] / n, 2),
    "avg_latency_seconds": round(agg["total_elapsed"] / n, 4),
    "p50_latency_seconds": runner._percentile(latency_list, 0.5),
    "p99_latency_seconds": runner._percentile(latency_list, 0.99),
}
return {
    "summary": agg,
    "macro_metrics": macro_metrics,
    "efficiency": efficiency,
    "detailed": detailed,
    "reference": BENCHMARK_REGISTRY.get(benchmark_key, {}).get("reference", ""),
}
if __name__ == "__main__":
    main()
// File: src/sakura_mcp/server.py
"""SakuraScholar MCP Server — 把检索/收藏/评测能力暴露为标准 MCP 工具。
岗位化改造 P0：对齐 JD 高频关键词「MCP / 工具调用 / 多智能体」，
可在 Claude Code / Cursor / 任意 MCP 客户端中直接调用：
    python mcp_server.py --transport stdio
```

```
    python mcp_server.py --transport sse --port 8766
工具清单：
    - search_papers    多通道检索（arXiv/OpenAlex/CrossRef/S2/本地语义 + LLM 判分）
    - get_paper        论文详情
    - get_references   参考文献（RefChain 引用方向）
    - get_citations    引用该文的论文（RefChain 被引方向）
    - save_favorite    收藏论文（按用户隔离）
    - list_favorites   列出收藏
    - run_eval         跑官方 benchmark 评测（F1/Recall）
"""
from __future__ import annotations
import asyncio
import concurrent.futures
import json
import sys
import tempfile
from pathlib import Path
from typing import Any, Dict, List, Optional
# 让 `python src/mcp_server.py` 与 `python -m mcp.server` 都能 import
ROOT = Path(__file__).resolve().parent.parent
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
try:
    from dotenv import load_dotenv
    load_dotenv(ROOT.parent / ".env")
except Exception:
    pass
from mcp.server.mcpserver import MCPServer
from agent.collections import add_favorite, list_favorites as list_favs
from agent.orchestrator import Orchestrator
from llm.router import get_default_router
mcp = MCPServer("SakuraScholar", instructions=(
    "科研场景学术论文检索系统。搜索论文用 search_papers；"
    "查详情/引文用 get_paper/get_references/get_citations；"
    "收藏与个人库用 save_favorite/list_favorites；"
    "跑官方测试集评测用 run_eval。"
))
# 与 Web 搜索一致的引擎配置：Web deadline 70s、首轮检索 + 判分
SEARCH_DEADLINE = 70.0
SEARCH_MAX_ROUNDS = 1
SEARCH_MAX_JUDGE = 40
SEARCH_LOCAL_TOP_K = 200
SEARCH_TIMEOUT = 95
def _get_llm(model: str = ""):
    """获取 LLM 实例；空则走默认路由（无可用端点时自动降级 MockLLM）。"""
    try:
        router = get_default_router()
        return router.get(model or None)
    except Exception:
        return None
def _run_with_timeout(fn, timeout: float) -> Any:
    """线程池 + 超时保护：外部 API 慢时返回友好错误而非挂死。"""
    executor = concurrent.futures.ThreadPoolExecutor(max_workers=1)
```

```
    try:
        return executor.submit(fn).result(timeout=timeout)
    except concurrent.futures.TimeoutError:
        raise TimeoutError(f"搜索超时（{timeout:.0f}s），外部 API 响应过慢，请稍后重试")
    finally:
        executor.shutdown(wait=False)
def _paper_to_markdown(p: Dict) -> str:
    title = p.get("title", "").strip() or "(无标题)"
    year = p.get("year") or ""
    venue = p.get("venue") or ""
    score = p.get("final_score", p.get("score", 0.0))
    pid = p.get("arxiv_id") or p.get("doi") or p.get("s2_corpus_id") or p.get("id", "")
    return f"- **{title}**（{year}）| {venue} | score={score:.3f} | id={pid}"
def _papers_summary(result: Dict) -> str:
    papers = result.get("papers", [])[:20]
    if not papers:
        return "未检索到相关论文。"
    lines = [f"返回 {len(papers)} 篇候选论文（按相关性排序）：", ""]
    for i, p in enumerate(papers, 1):
        lines.append(f"{i}. {_paper_to_markdown(p)}")
    lines.append("")
    metrics = result.get("metrics", {})
    lines.append(f"指标：API 调用 {metrics.get('api_calls', 0)} 次 / "
                 f"token {metrics.get('tokens', 0)} / 耗时 {metrics.get('latency', 0):.1f}s")
    innovations = result.get("innovations_used", [])
    if innovations:
        lines.append(f"启用的机制：{', '.join(innovations)}")
    return "\n".join(lines)
# ---------------------------------------------------------------- #
# 工具：检索
# ---------------------------------------------------------------- #
@mcp.tool()
def search_papers(query: str, top_k: int = 10, before_date: str = "",
                  llm_model: str = "") -> str:
    """多通道检索论文：查询理解 → arXiv/OpenAlex/CrossRef/S2/本地语义 5 通道召回
    → BGE 语义预筛 → LLM 批量判分 → 排序输出。
    Args:
        query: 学术查询（可复杂，如 "diffusion models for RL planning"）
        top_k: 返回条数上限
        before_date: 截止日期过滤（YYYY-MM-DD，空则不过滤）
        llm_model: LLM 模型名（空 = 默认路由；deepseek/qwen/glm/ollama 等）
    """
    orch = Orchestrator(
        llm_chat=_get_llm(llm_model),
        deadline_seconds=SEARCH_DEADLINE,
        max_rounds=SEARCH_MAX_ROUNDS,
        max_judge=SEARCH_MAX_JUDGE,
        local_top_k=SEARCH_LOCAL_TOP_K,
    )
    result = _run_with_timeout(
        lambda: orch.sync_run(query, before_date), SEARCH_TIMEOUT
    )
    result["papers"] = result.get("papers", [])[: max(1, top_k)]
```

```
    for p in result["papers"]:
        if p.get("final_score") is None:
            p["final_score"] = p.get("score", 0.0)
    summary = _papers_summary(result)
    detail = json.dumps(
        {"query": query, "papers": result["papers"],
         "metrics": result.get("metrics", {})},
        ensure_ascii=False, default=str,
    )
    return f"{summary}\n\n<detail>\n{detail}\n</detail>"
# ---------------------------------------------------------------- #
# 工具：论文详情与引文（RefChain）
# ---------------------------------------------------------------- #
def _fetch_papers(paper_id: str, method: str, use_real_api: bool) -> List[Dict]:
    """通过 Semantic Scholar 后端取详情/引文；离线时回退本地缓存。"""
    from retrieval import get_retrieval_backend
    backend = get_retrieval_backend("semantic_scholar", use_mock=not use_real_api)
    try:
        if method == "get_paper":
            paper = asyncio.run(backend.get_paper(paper_id))
            return [paper.to_dict()] if paper else []
        coro = backend.get_references if method == "get_references" else backend.get_citations
        papers = asyncio.run(coro(paper_id))
        return [p.to_dict() for p in papers]
    except Exception:
        if method == "get_paper":
            from agent.collections import get_cached_paper
            cached = get_cached_paper(paper_id)
            return [cached] if cached else []
        return []
@mcp.tool()
def get_paper(paper_id: str, use_real_api: bool = False) -> str:
    """获取论文详情（标题/摘要/年份/作者/来源）。
    Args:
        paper_id: arXiv ID 或 DOI 或 S2 corpusId
        use_real_api: 是否走真实 Semantic Scholar API（False 时用离线 Mock/缓存）
    """
    papers = _fetch_papers(paper_id, "get_paper", use_real_api)
    if not papers:
        return f"未找到论文：{paper_id}"
    return json.dumps(papers[0], ensure_ascii=False, indent=2, default=str)
@mcp.tool()
def get_references(paper_id: str, use_real_api: bool = False) -> str:
    """获取某篇论文的参考文献（RefChain 引用方向扩展）。
    Args:
        paper_id: arXiv ID 或 DOI 或 S2 corpusId
        use_real_api: 是否走真实 Semantic Scholar API
    """
    papers = _fetch_papers(paper_id, "get_references", use_real_api)
    if not papers:
        return f"未找到参考文献：{paper_id}"
    return json.dumps(papers, ensure_ascii=False, indent=2, default=str)
@mcp.tool()
```

```
def get_citations(paper_id: str, use_real_api: bool = False) -> str:
    """获取引用某篇论文的论文（RefChain 被引方向扩展）。
    Args:
        paper_id: arXiv ID 或 DOI 或 S2 corpusId
        use_real_api: 是否走真实 Semantic Scholar API
    """
    papers = _fetch_papers(paper_id, "get_citations", use_real_api)
    if not papers:
        return f"未找到引用该文的论文：{paper_id}"
    return json.dumps(papers, ensure_ascii=False, indent=2, default=str)
# ---------------------------------------------------------------- #
# 工具：收藏 / 个人库
# ---------------------------------------------------------------- #
@mcp.tool()
def save_favorite(paper_id: str, user_id: str = "mcp-user",
                  title: str = "", folder_id: str = "") -> str:
    """收藏一篇论文到个人库（按 user_id 隔离，可在 Web /app 收藏夹看到）。
    Args:
        paper_id: 论文 ID（arXiv/DOI/S2）
        user_id: 用户名（默认 mcp-user）
        title: 论文标题（可选，便于展示）
        folder_id: 目标文件夹 ID（空 = 默认文件夹）
    """
    meta = {"title": title} if title else None
    result = add_favorite(
        user_id, paper_id, paper_meta=meta, folder_id=folder_id or None
    )
    return json.dumps(result, ensure_ascii=False)
@mcp.tool()
def list_favorites(user_id: str = "mcp-user") -> str:
    """列出某个用户的收藏论文。
    Args:
        user_id: 用户名（默认 mcp-user）
    """
    data = list_favs(user_id)
    items = data.get("favorites", [])
    if not items:
        return "收藏夹为空。"
    lines = [f"共 {data.get('total', 0)} 篇收藏：", ""]
    for it in items:
        lines.append(f"- {it['paper_id']}（添加于 {it.get('added_at', '')}）")
    return "\n".join(lines)
# ---------------------------------------------------------------- #
# 工具：评测
# ---------------------------------------------------------------- #
@mcp.tool()
def run_eval(benchmark: str = "real_scholar", n: int = 3,
             real: bool = False, llm: str = "mock") -> str:
    """跑官方 benchmark 评测，返回 F1/Recall/Precision。
    Args:
        benchmark: 测试集名 real_scholar / auto_scholar / paper_finder / litqa2 / sparbench
        n: 评测条数（默认 3，快速验证）
        real: 是否真实检索（False 用离线语料，快且免费）
```

```
    llm: LLM 模型（mock 免费离线；deepseek/qwen/glm 等）
    """
    bench_map = {
        "real_scholar": "real_scholar", "auto_scholar": "auto_scholar",
        "paper_finder": "paper_finder", "litqa2": "litqa2", "sparbench": "sparbench",
    }
    if benchmark not in bench_map:
        return f"未知 benchmark：{benchmark}，可选 {list(bench_map)}"
    out = Path(tempfile.mkstemp(suffix=".json")[1])
    script = ROOT / "run_eval.py"
    args = [sys.executable, str(script), "--n", str(n),
            "--benchmarks", benchmark, "--llm", llm,
            "--output", str(out)]
    if real:
        args.append("--real")
    async def _run_proc():
        proc = await asyncio.create_subprocess_exec(
            *args, stdout=asyncio.subprocess.PIPE, stderr=asyncio.subprocess.STDOUT
        )
        return await proc.communicate()
    stdout, _ = asyncio.run(_run_proc())
    try:
        data = json.loads(out.read_text(encoding="utf-8"))
        report = data.get(benchmark, {}) or {}
        macro = report.get("macro_metrics", {})
        lines = [f"评测完成：{benchmark}（n={n}，llm={llm}）", ""]
        lines.append(f"- F1@10 = {macro.get('f1@10')} / Recall@20 = {macro.get('recall@20')} "
                     f"/ Precision@10 = {macro.get('precision@10')}")
        eff = report.get("efficiency", {})
        if eff:
            lines.append(f"- API 调用 {eff.get('avg_api_calls', 0)} 次 / "
                         f"token {eff.get('avg_tokens', 0)} / P50 {eff.get('p50_latency', 0)}s")
        return "\n".join(lines)
    except Exception as e:
        tail = (stdout or b"").decode("utf-8", "ignore")[-800:]
        return f"评测失败：{e}\n{tail}"
# ---------------------------------------------------------------- #
# CLI 入口
# ---------------------------------------------------------------- #
def main() -> None:
    import argparse
    parser = argparse.ArgumentParser(description="SakuraScholar MCP Server")
    parser.add_argument("--transport", choices=["stdio", "sse"], default="stdio")
    parser.add_argument("--port", type=int, default=8766, help="SSE 传输端口")
    args = parser.parse_args()
    if args.transport == "sse":
        import asyncio
        asyncio.run(mcp.run_sse_async(host="127.0.0.1", port=args.port))
    else:
        mcp.run(transport="stdio")
if __name__ == "__main__":
    main()
// File: src/web/app.py
```

```
"""
SakuraScholar Web 服务
基于 FastAPI 提供 REST API，供前端使用界面调用。
端点：
  GET  /              → 项目展示首页（🌸 HTML）
  GET  /app           → 搜索使用界面（🌸 HTML）
  POST /api/search    → 执行搜索（核心）
  GET  /api/paper/{id}  → 论文详情
  GET  /api/references/{id} → 参考文献
  GET  /api/citations/{id}  → 引用本文的论文
  POST /api/feedback  → 提交反馈事件
  GET  /api/feedback/report → 反馈统计报告
  GET  /api/health    → 健康检查
  GET  /api/benchmarks  → 列出可用 benchmark
  POST /api/eval      → 触发评测（后台）
  GET  /api/stats     → 系统统计
启动：
  uvicorn web.app:app --host 0.0.0.0 --port 8765 --reload
  或 python -m web.app
"""
from __future__ import annotations
import json
import logging
import os
import sys
from pathlib import Path
from typing import Any, Dict, List, Optional
# 路径修正
ROOT = Path(__file__).resolve().parent.parent
if str(ROOT) not in sys.path:
  sys.path.insert(0, str(ROOT))
from fastapi import FastAPI, HTTPException, Query
from fastapi.responses import HTMLResponse, JSONResponse, FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
from agent.feedback_loop import FeedbackCollector, FeedbackEvent, RewardComputer
from agent.orchestrator import DEFAULT_INNOVATION_FLAGS, Orchestrator
from evaluation.runner import BENCHMARK_REGISTRY, list_benchmarks
logger = logging.getLogger(__name__)
# ---------------------------------------------------------------- #
# 静态资源路径
# ---------------------------------------------------------------- #
WEB_DIR = Path(__file__).resolve().parent
STATIC_DIR = WEB_DIR / "static"
TEMPLATES_DIR = WEB_DIR / "templates"
# ---------------------------------------------------------------- #
# 请求/响应模型
# ---------------------------------------------------------------- #
class SearchRequest(BaseModel):
  query: str
  backends: Optional[List[str]] = None
  use_real_api: bool = False
  ablation: Optional[List[str]] = None  # ["qag", "mcs"]
```

```
    top_k: int = 0  # 0 = 不截断
    format: str = "json"  # json / three_part
    llm_model: str = ""  # 指定 LLM 模型名（对应 /api/llm/models 中的 name），空则用默认
class FeedbackRequest(BaseModel):
    query: str = ""
    paper_id: str
    event_type: str  # click / copy / cite / skip / hide / star
    star_value: Optional[int] = None
    user_id: str = "web-anonymous"
    session_id: str = "web"
class EvalRequest(BaseModel):
    benchmark: str = "sparbench"
    test_file: Optional[str] = None
    include_structural_check: bool = True
    use_real_api: bool = False  # 真实检索（CrossRef/S2/arXiv），否则 Mock
    limit: int = 0  # 0=全量；>0 只跑前 N 条（快速评测）
# ---------------------------------------------------------------- #
# 全局状态
# ---------------------------------------------------------------- #
_feedback_collector = FeedbackCollector()
_last_search_result: Optional[Dict[str, Any]] = None
# LLM 转发路由器（懒加载）
_llm_router = None
_LLM_CONFIG_FILE = ROOT / "data" / "llm_config.json"
def _load_llm_config() -> list:
    """读取持久化的 LLM 配置（register/register_preset 后重启不丢失）。"""
    try:
        if _LLM_CONFIG_FILE.exists():
            return json.loads(_LLM_CONFIG_FILE.read_text(encoding="utf-8"))
    except Exception:
        logger.exception("读取 LLM 配置失败")
    return []
def _save_llm_config() -> None:
    """把当前路由器的模型配置写入磁盘。"""
    router = _get_llm_router()
    cfg = []
    for ep in router._endpoints.values():  # type: ignore[attr-defined]
        if ep.name in ("mock",):
            continue
        cfg.append({
            "name": ep.name,
            "model": ep.model,
            "base_url": ep.base_url,
            "api_key": ep.api_key,
            "priority": ep.priority,
        })
    try:
        _LLM_CONFIG_FILE.write_text(
            json.dumps(cfg, ensure_ascii=False, indent=2), encoding="utf-8"
        )
    except Exception:
        logger.exception("保存 LLM 配置失败")
def _get_llm_router():
```

```
    """获取全局 LLM 路由器（首次调用时自动初始化）。"""
    global _llm_router
    if _llm_router is None:
        from llm.router import get_default_router
        _llm_router = get_default_router()
        # 恢复持久化的模型配置（env 启动的默认端点已存在则跳过）
        for item in _load_llm_config():
            try:
                if item.get("name") not in _llm_router._endpoints:  # type: ignore[attr-defined]
                    _llm_router.register(
                        name=item["name"],
                        model=item.get("model", ""),
                        base_url=item.get("base_url", ""),
                        api_key=item.get("api_key", ""),
                        enabled=bool(item.get("base_url")),
                        priority=item.get("priority", 0),
                    )
            except Exception:
                logger.exception("恢复 LLM 配置失败: %s", item.get("name"))
    return _llm_router
# ---------------------------------------------------------------- #
# FastAPI 应用
# ---------------------------------------------------------------- #
app = FastAPI(
    title="SakuraScholar",
    description="科研场景下复杂学术查询的智能论文搜索与推荐系统",
    version="1.0.0",
)
# 静态文件
if STATIC_DIR.exists():
    app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")
# ---------------------------------------------------------------- #
# 页面路由
# ---------------------------------------------------------------- #
@app.get("/", response_class=HTMLResponse)
async def index() -> HTMLResponse:
    """项目介绍首页。"""
    html_path = TEMPLATES_DIR / "index.html"
    if not html_path.exists():
        return HTMLResponse("<h1>templates/index.html not found</h1>", status_code=404)
    return HTMLResponse(html_path.read_text(encoding="utf-8"))
@app.get("/app", response_class=HTMLResponse)
async def app_page() -> HTMLResponse:
    """搜索使用界面。"""
    html_path = TEMPLATES_DIR / "app.html"
    if not html_path.exists():
        return HTMLResponse("<h1>templates/app.html not found</h1>", status_code=404)
    return HTMLResponse(html_path.read_text(encoding="utf-8"))
# ---------------------------------------------------------------- #
# API 端点
# ---------------------------------------------------------------- #
@app.get("/api/health")
async def health() -> Dict:
```

```
    return {"status": "ok", "service": "SakuraScholar", "version": "1.0.0"}
@app.get("/api/innovations")
async def get_innovations() -> Dict:
    """返回 15 个创新模块及其开关状态。"""
    return {
        "innovations": [
            {"key": k, "enabled": v, "name": _INNOVATION_NAMES.get(k, k)}
            for k, v in DEFAULT_INNOVATION_FLAGS.items()
        ]
    }
@app.post("/api/search")
def search(req: SearchRequest) -> Dict[str, Any]:
    """执行搜索（核心端点，同步以兼容 orchestrator 内部的 asyncio.run）。"""
    global _last_search_result
    import concurrent.futures
    # 获取 LLM 实例：优先用请求指定的模型，否则用默认路由器
    llm = None
    if req.llm_model:
        router = _get_llm_router()
        llm = router.get(req.llm_model)
    # SPAR2 引擎：始终走真实多通道检索（arXiv/OpenAlex/CrossRef/本地语义），
    # req.backends / req.use_real_api / req.ablation 为兼容旧接口保留（不生效）。
    # Web 模式 deadline 60s（评测模式用 170s），保证用户不会等太久
    # Web 模式：保留原 SPAR2 体系，但小服务器上只跑第一轮（含五通道 + 判分），
    # 第二轮/RefChain 留给评测模式；deadline 70s + 单次 LLM 30s 硬超时，
    # 保证在 Cloudflare 100s 内稳定返回
    orch = Orchestrator(llm_chat=llm, deadline_seconds=70.0, max_rounds=1,
                        max_judge=40, local_top_k=200)
    # 超时保护：外部 API 可能很慢，最多等 75 秒
    SEARCH_TIMEOUT = 95
    # 不用 with 语句，避免超时后等待后台线程结束才返回响应
    executor = concurrent.futures.ThreadPoolExecutor(max_workers=1)
    try:
        future = executor.submit(orch.sync_run, req.query)
        result = future.result(timeout=SEARCH_TIMEOUT)
    except concurrent.futures.TimeoutError:
        logger.warning("Search timed out after %ds for query: %s", SEARCH_TIMEOUT, req.query)
        executor.shutdown(wait=False)  # 不等待，立即返回 504
        raise HTTPException(
            status_code=504,
            detail=f"搜索超时（{SEARCH_TIMEOUT}s），外部 API 响应过慢。请稍后重试或缩小查询范
围。"
        )
    except Exception as e:
        logger.exception("search failed")
        executor.shutdown(wait=False)
        raise HTTPException(status_code=500, detail=str(e))
    finally:
        # 正常情况下等 future 完成（已经完成所以不阻塞）
        try:
            executor.shutdown(wait=False)
        except Exception:
            pass
```

```
    try:
        # top_k 截断
        if req.top_k and req.top_k > 0:
            result["papers"] = result["papers"][: req.top_k]
        # SPAR2 引擎只输出 score，前端统一读 final_score（展示/星图用）
        for p in result.get("papers", []):
            if p.get("final_score") is None:
                p["final_score"] = p.get("score", 0.0)
            p["select_score"] = None
            p["rerank_score"] = None
        # 三件套
        if req.format == "three_part":
            from agent.graph_synthesizer import generate_three_part_output, records_to_nodes
            from data.schemas import RelationshipEdge
            nodes = records_to_nodes(result["papers"])
            edges = [RelationshipEdge(**e) for e in result.get("relationships", [])]
            bundle = generate_three_part_output(req.query, nodes, edges)
            result["three_part"] = bundle
        _last_search_result = result
        return result
    except Exception as e:
        logger.exception("search post-processing failed")
        raise HTTPException(status_code=500, detail=str(e))
@app.get("/api/paper/{paper_id:path}")
def get_paper(paper_id: str, use_real_api: bool = Query(False)) -> Dict:
    """获取论文详情。"""
    from retrieval import get_retrieval_backend
    backend = get_retrieval_backend("semantic_scholar", use_mock=not use_real_api)
    import asyncio
    paper = asyncio.run(backend.get_paper(paper_id))
    if not paper:
        raise HTTPException(status_code=404, detail="Paper not found")
    return paper.to_dict()
@app.get("/api/references/{paper_id:path}")
def get_references(paper_id: str, use_real_api: bool = Query(False)) -> List[Dict]:
    """获取参考文献。"""
    from retrieval import get_retrieval_backend
    backend = get_retrieval_backend("semantic_scholar", use_mock=not use_real_api)
    import asyncio
    papers = asyncio.run(backend.get_references(paper_id))
    return [p.to_dict() for p in papers]
@app.get("/api/citations/{paper_id:path}")
def get_citations(paper_id: str, use_real_api: bool = Query(False)) -> List[Dict]:
    """获取引用本文的论文。"""
    from retrieval import get_retrieval_backend
    backend = get_retrieval_backend("semantic_scholar", use_mock=not use_real_api)
    import asyncio
    papers = asyncio.run(backend.get_citations(paper_id))
    return [p.to_dict() for p in papers]
@app.post("/api/feedback")
async def submit_feedback(req: FeedbackRequest) -> Dict:
    """提交反馈事件。"""
    etype = req.event_type.lower()
```

```
    if etype == "click":
        _feedback_collector.record_click(req.query, req.paper_id, user_id=req.user_id, session_id=req.session_id)
    elif etype == "copy":
        _feedback_collector.record_copy(req.query, req.paper_id, user_id=req.user_id, session_id=req.session_id)
    elif etype == "cite":
        _feedback_collector.record_cite(req.query, req.paper_id, user_id=req.user_id, session_id=req.session_id)
    elif etype == "skip":
        _feedback_collector.record_skip(req.query, req.paper_id, user_id=req.user_id, session_id=req.session_id)
    elif etype == "hide":
        _feedback_collector.record_hide(req.query, req.paper_id, user_id=req.user_id, session_id=req.session_id)
    elif etype == "star":
        _feedback_collector.record_star(req.query, req.paper_id, star=req.star_value or 5, user_id=req.user_id,
session_id=req.session_id)
    else:
        raise HTTPException(status_code=400, detail=f"Unknown event_type: {etype}")
    # 自动落盘
    _feedback_collector.flush_to_file(str(ROOT / "data" / "feedback.jsonl"))
    return {"status": "ok", "total_events": len(_feedback_collector.events) + _count_feedback_file()}
@app.get("/api/feedback/report")
async def feedback_report() -> Dict:
    """反馈统计报告。"""
    fb_path = ROOT / "data" / "feedback.jsonl"
    if not fb_path.exists():
        return {"total_events": 0, "message": "No feedback yet"}
    events = FeedbackCollector.load_from_file(str(fb_path))
    computer = RewardComputer()
    per_paper = computer.compute_per_paper(events)
    per_query = computer.compute_per_query(events)
    car = computer.citation_adoption_rate(events)
    return {
        "total_events": len(events),
        "unique_papers": len(per_paper),
        "unique_queries": len(per_query),
        "citation_adoption_rate": round(car, 4),
        "per_paper_reward": per_paper,
        "per_query_reward": per_query,
    }
@app.get("/api/benchmarks")
async def benchmarks() -> List[Dict]:
    """列出可用 benchmark。"""
    return list_benchmarks()
@app.post("/api/eval")
def run_eval(req: EvalRequest) -> Dict:
    """触发评测。
    同步端点：FastAPI 自动放到线程池跑，避免 async 端点里调
    orchestrator.run() 内部的 asyncio.run() 报 "cannot be called
    from a running event loop"。
    """
    from evaluation.runner import EvaluationRunner
    test_file = req.test_file
    if not test_file and req.benchmark:
        info = BENCHMARK_REGISTRY.get(req.benchmark)
        if not info:
```

```
            raise HTTPException(status_code=400, detail=f"Unknown benchmark: {req.benchmark}")
        test_file = info["path"]
        if not Path(test_file).exists():
            raise HTTPException(status_code=404, detail=f"Benchmark file not found: {test_file}")
    if not test_file or not Path(test_file).exists():
        raise HTTPException(status_code=400, detail="test_file not found")
    runner = EvaluationRunner(use_real_api=req.use_real_api, limit=req.limit)
    report = runner.evaluate(
        test_file=test_file,
        benchmark_name=req.benchmark or Path(test_file).stem,
        include_structural_check=req.include_structural_check,
    )
    return report
# ---------------------------------------------------------------------- #
# LLM 模型路由管理 API
# ---------------------------------------------------------------------- #
class LLMRegisterRequest(BaseModel):
    name: str
    model: str = "qwen3:8b"
    base_url: str = "http://localhost:11434/v1"
    api_key: str = ""
    enabled: bool = True
    priority: int = 0
class LLMTestRequest(BaseModel):
    name: str
    prompt: str = "你好，用一句话介绍你自己"
@app.get("/api/llm/models")
async def list_llm_models() -> Dict:
    """列出所有已注册的 LLM 模型端点。"""
    router = _get_llm_router()
    return {"models": router.list_endpoints()}
@app.post("/api/llm/register")
async def register_llm_model(req: LLMRegisterRequest, token: str = "") -> Dict:
    """注册一个新的 LLM 模型端点（仅管理员可操作，保护已有 key 配置）。"""
    if not _is_admin(token):
        raise HTTPException(status_code=403, detail="仅管理员可配置大模型")
    router = _get_llm_router()
    router.register(
        name=req.name,
        model=req.model,
        base_url=req.base_url,
        api_key=req.api_key or os.getenv("OPENAI_API_KEY", ""),
        enabled=req.enabled,
        priority=req.priority,
    )
    _save_llm_config()
    return {"status": "ok", "models": router.list_endpoints()}
@app.post("/api/llm/unregister/{name}")
async def unregister_llm_model(name: str, token: str = "") -> Dict:
    """移除一个 LLM 模型端点（仅管理员）。"""
    if not _is_admin(token):
        raise HTTPException(status_code=403, detail="仅管理员可删除模型")
    router = _get_llm_router()
```

```
    router.unregister(name)
    _save_llm_config()
    return {"status": "ok", "models": router.list_endpoints()}
@app.post("/api/llm/test")
async def test_llm_model(req: LLMTestRequest) -> Dict:
    """测试指定 LLM 模型端点是否可用。"""
    import time
    router = _get_llm_router()
    try:
        llm = router.get(req.name)
        t0 = time.time()
        result = llm.complete(req.prompt)
        elapsed = round(time.time() - t0, 3)
        return {
            "status": "ok",
            "name": req.name,
            "response": result[:300],
            "elapsed_seconds": elapsed,
        }
    except Exception as e:
        logger.exception("LLM 测试失败: %s", e)
        return {"status": "error", "name": req.name, "error": str(e)}
@app.get("/api/stats")
async def stats() -> Dict:
    """系统统计。"""
    return {
        "feedback_events": _count_feedback_file(),
        "innovations": {k: v for k, v in DEFAULT_INNOVATION_FLAGS.items()},
        "benchmarks": list_benchmarks(),
        "last_search_papers": len(_last_search_result["papers"]) if _last_search_result else 0,
    }
# ---------------------------------------------------------------------- #
# LLM 预设模型（多模型一键接入）
# ---------------------------------------------------------------------- #
@app.get("/api/llm/presets")
async def list_llm_presets() -> Dict:
    """列出所有预设 LLM 提供商（含聚合路由 OpenRouter/SiliconFlow）。"""
    from llm.openai_compat import list_presets, AGGREGATOR_MODELS
    presets = list_presets()
    # 标记是否为聚合路由
    for p in presets:
        p["is_aggregator"] = p["key"] in ("openrouter", "siliconflow")
        if p["is_aggregator"]:
            p["models"] = AGGREGATOR_MODELS.get(p["key"], [])
    return {"presets": presets}
@app.get("/api/llm/aggregator_models/{provider}")
async def list_aggregator_models(provider: str) -> Dict:
    """获取聚合路由（OpenRouter/SiliconFlow）的可用模型列表。"""
    from llm.openai_compat import AGGREGATOR_MODELS
    if provider not in AGGREGATOR_MODELS:
        raise HTTPException(status_code=400, detail=f"未知聚合路由: {provider}")
    return {"provider": provider, "models": AGGREGATOR_MODELS[provider]}
class LLMPresetRegisterRequest(BaseModel):
```

```
    provider: str      # openai/deepseek/qwen_api/glm/moonshot/ollama
    api_key: str = ""  # 云端 API 必填，Ollama 留空
    model: str = ""    # 可覆盖预设默认模型
    priority: int = 10
@app.post("/api/llm/register_preset")
async def register_preset(req: LLMPresetRegisterRequest, token: str = "") -> Dict:
    """一键注册预设模型（自动填 base_url + 默认 model；仅管理员）。"""
    if not _is_admin(token):
        raise HTTPException(status_code=403, detail="仅管理员可配置大模型")
    from llm.openai_compat import PRESETS
    preset = PRESETS.get(req.provider)
    if not preset:
        raise HTTPException(status_code=400, detail=f"未知 provider: {req.provider}")
    router = _get_llm_router()
    model = req.model or preset["model"]
    # ★ 前端不传 key 时自动读取后端 .env 的 key（避免"配好了却 401/离线"）
    api_key = req.api_key or os.getenv(preset.get("env_key", ""), "")
    router.register(
        name=req.provider,
        model=model,
        base_url=preset["base_url"],
        api_key=api_key,
        enabled=True,
        priority=req.priority,
    )
    _save_llm_config()
    return {
        "status": "ok",
        "registered": {
            "name": req.provider,
            "model": model,
            "base_url": preset["base_url"],
            "api_key_loaded": bool(api_key),
        },
        "models": router.list_endpoints(),
    }
# ---------------------------------------------------------------- #
# 收藏夹 / 已阅 / BibTeX 导出（让反馈真正有用）
# ---------------------------------------------------------------- #
def _load_feedback_events() -> list:
    """加载所有反馈事件。"""
    fb_path = ROOT / "data" / "feedback.jsonl"
    if not fb_path.exists():
        return []
    out = []
    for line in fb_path.read_text(encoding="utf-8").splitlines():
        line = line.strip()
        if line:
            try:
                out.append(json.loads(line))
            except json.JSONDecodeError:
                continue
    return out
```

```
def _user_papers_by_event(event_type: str, user_id: str = "web-anonymous") -> list:
    """按事件类型聚合用户的论文 id 列表（去重，按时间倒序）。"""
    events = _load_feedback_events()
    seen = {}
    for e in events:
        if e.get("event_type") == event_type and e.get("user_id", "web-anonymous") == user_id:
            pid = e.get("paper_id", "")
            if pid and pid not in seen:
                seen[pid] = e.get("timestamp", 0)
    return sorted(seen.keys(), key=lambda p: seen[p], reverse=True)
@app.get("/api/favorites")
async def get_favorites(user_id: str = "web-anonymous", folder_id: str = "") -> Dict:
    """获取用户的收藏夹（带论文详情，支持按文件夹过滤）。
    folder_id 参数语义：
     - 空/不传 → 不过滤，返回全部
     - "unfiled" → 仅返回未分类
     - "fXXX"  → 仅返回该文件夹下的
    """
    from agent.collections import list_favorites, get_cached_paper
    from retrieval import get_retrieval_backend
    # 归一化："unfiled" → ""（未分类）；空字符串且未显式传参 → None（不过滤）
    if folder_id == "unfiled":
        filter_fid = ""  # 未分类
    elif folder_id:
        filter_fid = folder_id
    else:
        filter_fid = None  # 不过滤
    result = list_favorites(user_id if user_id != "web-anonymous" else None,
                            folder_id=filter_fid)
    papers = []
    backend = get_retrieval_backend("semantic_scholar", use_mock=True)
    for item in result["favorites"]:
        pid = item["paper_id"]
        # 优先用缓存元数据
        cached = get_cached_paper(pid)
        if cached and cached.get("title"):
            # 补全缺失字段（旧缓存可能没有 url/authors/venue）
            if not cached.get("url"):
                if cached.get("arxiv_id"):
                    cached["url"] = f"https://arxiv.org/abs/{cached['arxiv_id']}"
                elif cached.get("doi"):
                    cached["url"] = f"https://doi.org/{cached['doi']}"
            cached.setdefault("authors", [])
            cached.setdefault("venue", "")
            papers.append({
                **cached,
                "paper_id": pid,
                "added_at": item.get("added_at"),
                "folder_id": item.get("folder_id"),
                "note": item.get("note", ""),
            })
            continue
        # 兜底：从 retrieval backend 获取
```

```
        try:
            p = await backend.get_paper(pid)
            if p:
                papers.append({**p.to_dict(),
                               "paper_id": pid,
                               "added_at": item.get("added_at"),
                               "folder_id": item.get("folder_id"),
                               "note": item.get("note", "")})
            else:
                papers.append({"arxiv_id": pid, "title": pid,
                               "paper_id": pid,
                               "added_at": item.get("added_at"),
                               "note": "详情不可用"})
        except Exception as e:
            logger.warning("get_paper(%s) 失败: %s", pid, e)
            papers.append({"arxiv_id": pid, "title": pid,
                           "paper_id": pid,
                           "added_at": item.get("added_at"),
                           "note": "详情不可用"})
    return {"favorites": papers, "total": result["total"]}
# ---------------------------------------------------------------- #
# 收藏夹批量导入（Zotero / BibTeX / RIS / CSV / EndNote JSON）
# 注意：必须注册在 /api/favorites/{paper_id:path} 通配路由之前，
# 否则 POST /api/favorites/import 会被当成收藏一篇 id=import 的论文。
# ---------------------------------------------------------------- #
class FavoriteImportItem(BaseModel):
    """单条导入项。前端解析后逐条传入。"""
    id: str = ""              # arxiv_id / doi / 自定义 id
    title: str = ""
    authors: List[str] = []
    year: Optional[int] = None
    venue: str = ""
    doi: str = ""
    url: str = ""
    abstract: str = ""
    note: str = ""
class FavoriteImportRequest(BaseModel):
    """批量导入请求。"""
    items: List[FavoriteImportItem]
    source: str = "zotero"   # zotero / bibtex / ris / csv / json
    user_id: str = "web-anonymous"
    session_id: str = "web-import"
@app.post("/api/favorites/import")
async def import_favorites(req: FavoriteImportRequest) -> Dict:
    """批量导入论文到收藏夹。
    前端把 BibTeX / RIS / CSV / Zotero JSON 解析成统一的 items 列表后调用本接口。
    每条 item 会：
      1. 写入 collections 存储（按 user_id 隔离）
      2. 缓存论文详情到 paper_cache.json + retrieval backend mock_corpus
    """
    from agent.collections import add_favorite as _add, _cache_paper_meta
    from retrieval import get_retrieval_backend
    from retrieval.base import RawPaper
```

```
    backend = get_retrieval_backend("semantic_scholar", use_mock=True)
    ok_count = 0
    skipped = 0
    username = req.user_id if req.user_id != "web-anonymous" else None
    for item in req.items:
        pid = item.id or item.doi or item.title
        if not pid:
            skipped += 1
            continue
        # 1) 写入 collections 存储
        paper_meta = {
            "title": item.title or pid,
            "authors": item.authors or [],
            "year": item.year,
            "venue": item.venue or "",
            "doi": item.doi or "",
            "url": item.url or "",
            "abstract": item.abstract or "",
            "arxiv_id": pid if "arxiv" in (item.url or "").lower() else "",
            "id": pid,
        }
        _add(username, pid, paper_meta=paper_meta)
        # 2) 缓存到 retrieval backend mock_corpus（供 BibTeX 导出、关系图查询使用）
        if item.title or item.authors:
            try:
                paper = RawPaper(
                    title=item.title or pid,
                    abstract=item.abstract or "",
                    paper_id=pid,
                    doi=item.doi or "",
                    year=item.year or 0,
                    authors=item.authors or [],
                    venue=item.venue or "",
                    url=item.url or "",
                    source="import",
                )
                backend._mock_corpus[pid] = paper
                try:
                    from retrieval.base import _GLOBAL_PAPER_CORPUS
                    _GLOBAL_PAPER_CORPUS[pid] = paper
                except Exception:
                    pass
            except Exception as e:
                logger.warning("缓存导入论文失败: %s", e)
        ok_count += 1
    return {
        "status": "ok",
        "imported": ok_count,
        "skipped": skipped,
        "total": len(req.items),
    }
@app.post("/api/favorites/{paper_id:path}")
async def add_favorite(paper_id: str, body: Dict = None, user_id: str = "web-anonymous") -> Dict:
```

```
    """添加收藏。body: {paper_meta?: {...}, folder_id?: "f1"}"""
    from agent.collections import add_favorite as _add
    body = body or {}
    paper_meta = body.get("paper_meta") if isinstance(body, dict) else None
    folder_id = body.get("folder_id") if isinstance(body, dict) else None
    return _add(user_id if user_id != "web-anonymous" else None,
                paper_id, paper_meta=paper_meta, folder_id=folder_id)
@app.delete("/api/favorites/{paper_id:path}")
async def remove_favorite(paper_id: str, user_id: str = "web-anonymous") -> Dict:
    """从收藏夹移除（真正从存储中删除）。"""
    from agent.collections import remove_favorite as _remove
    return _remove(user_id if user_id != "web-anonymous" else None, paper_id)
@app.get("/api/favorites/check/{paper_id:path}")
async def check_favorited(paper_id: str, user_id: str = "web-anonymous") -> Dict:
    """检查某篇论文是否已收藏。"""
    from agent.collections import is_favorited
    return {"favorited": is_favorited(user_id if user_id != "web-anonymous" else None, paper_id)}
@app.get("/api/read_papers")
async def get_read_papers(user_id: str = "web-anonymous") -> Dict:
    """获取已阅论文 id 列表。"""
    from agent.collections import list_read_papers
    return list_read_papers(user_id if user_id != "web-anonymous" else None)
@app.post("/api/read_papers/{paper_id:path}")
async def add_read_paper(paper_id: str, user_id: str = "web-anonymous") -> Dict:
    """标记为已阅。"""
    from agent.collections import add_read_paper as _add
    return _add(user_id if user_id != "web-anonymous" else None, paper_id)
@app.delete("/api/read_papers/{paper_id:path}")
async def remove_read_paper(paper_id: str, user_id: str = "web-anonymous") -> Dict:
    """取消已阅。"""
    from agent.collections import remove_read_paper as _remove
    return _remove(user_id if user_id != "web-anonymous" else None, paper_id)
@app.get("/api/read_papers/check/{paper_id:path}")
async def check_read(paper_id: str, user_id: str = "web-anonymous") -> Dict:
    """检查某篇论文是否已阅。"""
    from agent.collections import is_read
    return {"read": is_read(user_id if user_id != "web-anonymous" else None, paper_id)}
# ---------------------------------------------------------------- #
# 收藏夹文件夹管理
# ---------------------------------------------------------------- #
@app.get("/api/folders")
async def list_folders(user_id: str = "web-anonymous") -> Dict:
    """列出所有文件夹。"""
    from agent.collections import list_folders as _list
    return _list(user_id if user_id != "web-anonymous" else None)
@app.get("/api/folders/default")
async def get_default_folder(user_id: str = "web-anonymous") -> Dict:
    """获取默认收藏文件夹。"""
    from agent.collections import get_default_folder as _get
    return _get(user_id if user_id != "web-anonymous" else None)
@app.post("/api/folders")
async def create_folder(body: Dict, user_id: str = "web-anonymous") -> Dict:
    """创建文件夹。body: {name: str, description?: str}"""
```

```
    from agent.collections import create_folder as _create
    name = body.get("name", "") if isinstance(body, dict) else ""
    desc = body.get("description", "") if isinstance(body, dict) else ""
    return _create(user_id if user_id != "web-anonymous" else None, name, desc)
@app.put("/api/folders/default")
async def set_default_folder(body: Dict, user_id: str = "web-anonymous") -> Dict:
    """设置默认收藏文件夹。body: {folder_id: str}（"" 表示未分类）。"""
    from agent.collections import set_default_folder as _set
    folder_id = body.get("folder_id", "") if isinstance(body, dict) else ""
    return _set(user_id if user_id != "web-anonymous" else None, folder_id)
@app.delete("/api/folders/{folder_id}")
async def delete_folder(folder_id: str, user_id: str = "web-anonymous") -> Dict:
    """删除文件夹。"""
    from agent.collections import delete_folder as _delete
    return _delete(user_id if user_id != "web-anonymous" else None, folder_id)
@app.put("/api/folders/{folder_id}")
async def rename_folder(folder_id: str, body: Dict, user_id: str = "web-anonymous") -> Dict:
    """重命名文件夹。body: {name: str}"""
    from agent.collections import rename_folder as _rename
    name = body.get("name", "") if isinstance(body, dict) else ""
    return _rename(user_id if user_id != "web-anonymous" else None, folder_id, name)
@app.post("/api/folders/{folder_id}/papers/{paper_id:path}")
async def move_paper_to_folder(folder_id: str, paper_id: str,
                               user_id: str = "web-anonymous") -> Dict:
    """把论文移到指定文件夹。"""
    from agent.collections import move_paper_to_folder as _move
    # folder_id 为 "unfiled" 时表示移到未分类
    fid = None if folder_id == "unfiled" else folder_id
    return _move(user_id if user_id != "web-anonymous" else None, paper_id, fid)
# ---------------------------------------------------------------- #
# 使用文档（管理员可编辑）
# ---------------------------------------------------------------- #
_DOCS_FILE = ROOT / "data" / "docs_content.json"
_DEFAULT_DOCS = [
    {
      "title": "快速上手",
      "body": "在搜索框输入你想查的问题（可以带限定，如「RAG 2023 年之后的综述」），点搜索。
\n"
            "系统会同时从 arXiv、OpenAlex、CrossRef、Semantic Scholar 和本地语义库检索，"
            "再用大模型逐篇判断相关性，通常 1-2 分钟出结果。\n"
            "如果提示超时，多半是外部接口慢，稍后重试或把问题说简单一点。",
    },
    {
      "title": "收藏夹",
      "body": "点卡片上的「收藏」把论文存起来。\n"
            "在「管理文件夹」里可以新建文件夹，并选一个「默认收藏文件夹」——"
            "之后新收藏的论文会自动进这个文件夹，不用每次手动移动。\n"
            "收藏列表里每篇论文都有下拉菜单，可以随时移到别的文件夹或移回未分类。",
    },
    {
      "title": "导入 / 导出",
      "body": "导入：从 Zotero / EndNote / Mendeley 等软件导出 BibTeX (.bib)、RIS、CSV 或 JSON 文
件，"
```

```
            "拖进「导入 / 导出」页的框里即可，导入的论文进当前账号的收藏夹。\n"
            "导出：先选导出范围（全部 / 某个文件夹），再点对应的格式按钮。"
            "在某个文件夹里导出时，文件名就是文件夹名（比如 tabpfn.bib）。",
    },
    {
        "title": "账号",
        "body": "点右上角「登录」注册或登录。\n"
            "登录后收藏、已阅、文件夹、导入导出都按账号隔离，彼此看不到对方的数据；"
            "未登录时是匿名账号，数据只在这台浏览器上。\n"
            "已登录时再点右上角按钮进入「账号」面板，可以修改密码或退出登录。",
    },
    {
        "title": "设置",
        "body": "「设置 → 检索通道」：搜索固定使用 5 路检索，不用配置。\n"
            "同一页下方可以填 OpenAlex API Key / 邮箱和 Semantic Scholar Key（可选，额度更稳），"
            "填完点保存即可，不用改服务器文件。\n"
            "「⚙ 配置大模型」：预设里一键添加 DeepSeek / Ollama 等，或自定义 OpenAI 兼容端点。",
    },
    {
        "title": "常见问题",
        "body": "- 搜索超时：外部 API 慢或额度用完，稍后重试。\n"
            "- 文献星图没有连线：这批结果里论文之间关联弱，换个更具体的关键词。\n"
            "- 觉得字小：界面默认已放大 15%，需要再调告诉我即可。",
    },
]
def _load_docs() -> list:
    """读取可编辑的文档内容；文件不存在或损坏时用默认内容。"""
    if not _DOCS_FILE.exists():
        return list(_DEFAULT_DOCS)
    try:
        data = json.loads(_DOCS_FILE.read_text(encoding="utf-8"))
        if isinstance(data, list) and data:
            return data
    except (json.JSONDecodeError, OSError) as e:
        logger.warning("读取 docs_content.json 失败: %s", e)
    return list(_DEFAULT_DOCS)
def _is_admin(token: str) -> bool:
    """文档管理员：SAKURA_ADMIN_USERS / SAKURA_ADMIN_USER 指定的用户名。"""
    from agent.collections import verify_token
    username = verify_token(token) if token else None
    if not username:
        return False
    admins = [u.strip() for u in os.getenv("SAKURA_ADMIN_USERS", "").split(",") if u.strip()]
    admins.append(os.getenv("SAKURA_ADMIN_USER", "admin"))
    return username in admins
# ---------------------------------------------------------------- #
# 外部 API 配置（OpenAlex / Semantic Scholar，设置面板可改）
# ---------------------------------------------------------------- #
@app.get("/api/settings/api")
async def get_api_settings(token: str = "") -> Dict:
    """返回 API 配置（key 脱敏）+ 是否已登录。"""
    from agent.api_settings import load_api_config, mask_key
    from agent.collections import verify_token
```

```
    cfg = load_api_config()
    return {
        "status": "ok",
        "logged_in": bool(verify_token(token)) if token else False,
        "openalex_key": mask_key(cfg.get("openalex_key")),
        "openalex_email": cfg.get("openalex_email", ""),
        "s2_key": mask_key(cfg.get("s2_key")),
    }
@app.put("/api/settings/api")
async def update_api_settings(req: Dict, token: str = "") -> Dict:
    """保存 API 配置（需登录）。body: {openalex_key?, openalex_email?, s2_key?}"""
    from agent.api_settings import save_api_config
    from agent.collections import verify_token
    if not _is_admin(token):
        raise HTTPException(status_code=403, detail="仅管理员可修改 API 配置")
    if not isinstance(req, dict):
        raise HTTPException(status_code=400, detail="请求格式错误")
    try:
        cfg = save_api_config(
            openalex_key=req.get("openalex_key"),
            openalex_email=req.get("openalex_email"),
            s2_key=req.get("s2_key"),
        )
    except RuntimeError as e:
        raise HTTPException(status_code=500, detail=str(e)) from e
    return {"status": "ok", "configured": {k: bool(v) for k, v in cfg.items()}}
@app.on_event("startup")
async def _seed_admin_user() -> None:
    """启动时预置内置管理员账号（默认 admin / admin123，可用 .env 覆盖）。"""
    try:
        from agent.collections import ensure_admin_user
        ensure_admin_user(
            username=os.getenv("SAKURA_ADMIN_USER", "admin"),
            password=os.getenv("SAKURA_ADMIN_PASSWORD", "admin123"),
        )
    except Exception as e:  # noqa: BLE001
        logger.warning("预置管理员账号失败: %s", e)
@app.get("/api/docs")
async def get_docs(token: str = "") -> Dict:
    """获取使用文档内容；返回当前用户是否有编辑权限。"""
    return {"status": "ok", "sections": _load_docs(), "can_edit": _is_admin(token)}
@app.put("/api/docs")
async def update_docs(req: Dict, token: str = "") -> Dict:
    """管理员保存文档内容。body: {sections: [{title, body}]}"""
    if not _is_admin(token):
        raise HTTPException(status_code=403, detail="仅管理员可编辑文档")
    sections = req.get("sections") if isinstance(req, dict) else None
    if not isinstance(sections, list) or not sections:
        raise HTTPException(status_code=400, detail="sections 不能为空")
    cleaned = []
    for s in sections:
        if not isinstance(s, dict):
            continue
```

```
        title = str(s.get("title", "")).strip()
        body = str(s.get("body", "")).strip()
        if title or body:
            cleaned.append({"title": title or "未命名", "body": body})
    if not cleaned:
        raise HTTPException(status_code=400, detail="文档内容不能为空")
    try:
        _DOCS_FILE.parent.mkdir(parents=True, exist_ok=True)
        _DOCS_FILE.write_text(
            json.dumps(cleaned, ensure_ascii=False, indent=2), encoding="utf-8"
        )
    except OSError as e:
        logger.error("保存 docs_content.json 失败: %s", e)
        raise HTTPException(status_code=500, detail="保存失败")
    return {"status": "ok", "sections": cleaned}
# ---------------------------------------------------------------- #
# 用户认证（本地账号 + 密码 hash）
# ---------------------------------------------------------------- #
class AuthRequest(BaseModel):
    username: str
    password: str
@app.post("/api/auth/register")
async def auth_register(req: AuthRequest) -> Dict:
    """注册新用户。"""
    from agent.collections import register_user
    result = register_user(req.username, req.password)
    if result["status"] == "error":
        raise HTTPException(status_code=400, detail=result["detail"])
    return result
@app.post("/api/auth/login")
async def auth_login(req: AuthRequest) -> Dict:
    """登录。"""
    from agent.collections import login_user
    # 安全网：管理员账号被误删时，登录尝试会自动用 .env 配置重建（幂等）
    try:
        from agent.collections import ensure_admin_user
        ensure_admin_user(
            username=os.getenv("SAKURA_ADMIN_USER", "admin"),
            password=os.getenv("SAKURA_ADMIN_PASSWORD", "admin123"),
        )
    except Exception as e:  # noqa: BLE001
        logger.warning("重建管理员账号失败: %s", e)
    result = login_user(req.username, req.password)
    if result["status"] == "error":
        raise HTTPException(status_code=401, detail=result["detail"])
    return result
class ChangePasswordRequest(BaseModel):
    token: str = ""
    old_password: str = ""
    new_password: str = ""
@app.post("/api/auth/change-password")
async def auth_change_password(req: ChangePasswordRequest) -> Dict:
    """修改密码。"""
```

```
    from agent.collections import change_password
    result = change_password(req.token, req.old_password, req.new_password)
    if result["status"] == "error":
        raise HTTPException(status_code=400, detail=result["detail"])
    return result
@app.post("/api/auth/logout")
async def auth_logout(token: str = "") -> Dict:
    """登出。"""
    from agent.collections import logout_user
    return logout_user(token)
@app.get("/api/auth/me")
async def auth_me(token: str = "") -> Dict:
    """获取当前登录用户。"""
    from agent.collections import verify_token
    username = verify_token(token) if token else None
    if username:
        return {"logged_in": True, "username": username}
    return {"logged_in": False, "username": ""}
@app.get("/api/export/bibtex")
async def export_bibtex(paper_id: str) -> Dict:
    """导出指定论文的 BibTeX 引用格式。"""
    import re
    from agent.collections import get_cached_paper
    from retrieval import get_retrieval_backend
    # 优先从缓存获取（收藏夹保存的元数据）
    d = get_cached_paper(paper_id)
    if not d or not d.get("title"):
        # 回退到 retrieval backend
        backend = get_retrieval_backend("semantic_scholar", use_mock=True)
        paper = await backend.get_paper(paper_id)
        if paper:
            d = paper.to_dict()
    if not d or not d.get("title"):
        raise HTTPException(status_code=404, detail="Paper not found")
    # cite key: 第一作者姓 + 年份 + 标题首词（只保留字母数字）
    authors_list = d.get("authors") or []
    first_author = authors_list[0] if authors_list else "Anonymous"
    last_name = re.sub(r'[^a-z0-9]', '', first_author.split()[-1].lower()) if first_author else "anon"
    year = d.get("year") or 2024
    first_word = re.sub(r'[^a-z0-9]', '', (d.get("title") or "untitled").split()[0].lower())
    cite_key = f"{last_name or 'anon'}{year}{first_word or 'untitled'}"
    authors_str = " and ".join(authors_list) if authors_list else "Anonymous"
    # 构造 BibTeX（只输出非空字段）
    lines = [
        f"@article{{{cite_key},",
        f"  title     = {{{d.get('title', '')}}},",
        f"  author    = {{{authors_str}}},",
        f"  year      = {{{year}}},",
    ]
    if d.get("venue"):
        lines.append(f"  journal   = {{{d['venue']}}},")
    if d.get("arxiv_id"):
        lines.append(f"  eprint    = {{{d['arxiv_id']}}},")
```

```
    if d.get("doi"):
        lines.append(f"  doi       = {{{d['doi']}}},")
    # 构造 URL（回退推导）
    url = d.get("url") or ""
    if not url:
        if d.get("arxiv_id"):
            url = f"https://arxiv.org/abs/{d['arxiv_id']}"
        elif d.get("doi"):
            url = f"https://doi.org/{d['doi']}"
    if url:
        lines.append(f"  url       = {{{url}}},")
    lines.append("}")
    bibtex = "\n".join(lines)
    return {"bibtex": bibtex, "cite_key": cite_key}
# ---------------------------------------------------------------------- #
# 辅助
# ---------------------------------------------------------------------- #
def _count_feedback_file() -> int:
    fb_path = ROOT / "data" / "feedback.jsonl"
    if not fb_path.exists():
        return 0
    return sum(1 for line in fb_path.read_text(encoding="utf-8").splitlines() if line.strip())
# 创新机制中文名映射（SPAR2 常驻机制）
_INNOVATION_NAMES = {
    "spar2_bfs_iterative": "① 迭代检索（BFS 多轮）",
    "spar2_doc_query": "② 文档驱动查询生成",
    "spar2_refchain_bi": "③ RefChain 双向引文扩展",
    "spar2_bge_prescreen": "④ BGE 语义预筛 + 语义保底",
    "spar2_multi_channel": "⑤ 多通道检索",
    "spar2_deadline_filter": "⑥ 评测截止日期过滤",
}
# ---------------------------------------------------------------------- #
# 直接运行
# ---------------------------------------------------------------------- #
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        "web.app:app",
        host="0.0.0.0",
        port=8765,
        reload=True,
        log_level="info",
    )
```